



DENA-Architecture

Document meta-data

Author	DENA-NEU support office
Document Description	DENA Architecture

Version control

Versión	Date	Author	Description

TABLE OF CONTENTS

1	HIGH LEVEL VIEW OF DENA.....	4
1.1	DENA PREMISES	4
1.2	DENA CONCEPTS	5
1.3	DENA [DATA] INTERCHANGE MECHANISM HIGH LEVEL VIEW.....	6
1.3.1	SYNC [meta-data] about changes of [data] at the origin [admin]	6
1.3.2	Retrieve the full [data] from the [data origin] (admin).....	9
1.3.3	The COLD-START problem.....	2
1.4	PERSON SYNC	3
2	DETAILED ARCHIECTURE	4
2.1	SYNC AND RETRIEVE	1
2.2	PERSON SYNC	1
2.2.1	Push Sync	1
2.2.2	Pull Sync	1
3	SECURITY	3
3.1	[PERSON] AUTH WITH DENA-APP: GILTZA OAUTH TOKEN.....	4
3.2	DENA-APP AUTH WITH DENA-CORE: DENA-IDP OAUTH TOKEN	4
3.3	[ADMIN] AUTH WITH DENA-CORE: DENA-IDP OAUTH TOKEN.....	5
3.4	DENA-CORE AUTH WITH [ADMIN]: TARGET [ADMIN] PREFERRED [AUTH] SYSTEM (OAUTH, X-509 CERTIFICATE, USER/PASSWD...).....	5
4	SERVICES INTERNAL ARCHITECTURE	7
4.1	OVERALL VIEW	7
4.2	INTERCHANGED DATA STRUCTURES	9
4.2.1	[interop messages] used at REST-Services vs [model objects] used at JAVA-APIs	9
4.2.2	[interop message] components	13
5	DENA CONFIG	16
5.1	LABELING SYSTEM	16
5.1.1	Main Objects in the Labeling System.....	17
5.1.2	Java API	18
5.2	ORG CONFIG.....	20
5.3	INTEROP CONFIG	21
5.3.1	Data Type	21
5.3.2	Data Origin	21
5.4	APPVERSION CONFIG	24
6	BASIC DENA DATA-TYPES.....	25
6.1	FOUNDATIONAL DATA-TYPES	25
6.1.1	Boolean	25
6.1.2	Numbers.....	25
6.1.3	Enums.....	25
6.1.4	Language / Country.....	25
6.1.5	String	25
6.1.6	LanguageTexts.....	26
6.1.7	Date Time	26
6.1.8	Ranges	27
6.1.9	UID / OID	28
6.1.10	ID	28
6.1.11	Token.....	28

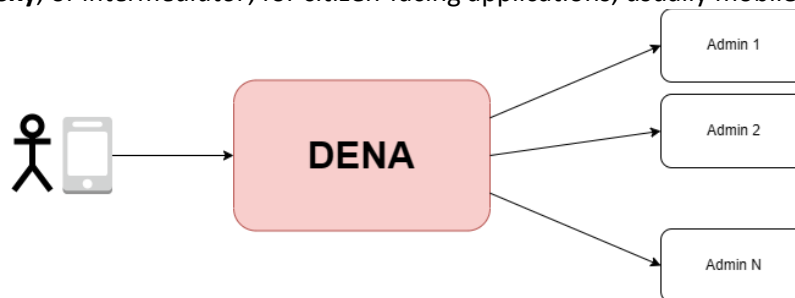
6.1.12	Urls	29
6.1.13	Money	30
6.1.14	Hash (digest)	31
6.1.15	UserAgent	32

1 HIGH LEVEL VIEW OF DENA

DENA is a **[person] ↔ [admin] interoperability** system. In other words, it supports interoperability between a [person] and an [administration], rather than interoperability between two administrations ([admin] ↔ [admin] interoperability)

The aim of DENA is to **provide the technical basis for applications that give citizens an aggregated view of their [data] across any [admin].**

DENA interop is based on **[services] exposed by the [admins]**. These services are called by DENA, which acts as a kind of **proxy**, or *intermediator*, for citizen-facing applications, usually mobile or web apps.



1.1 DENA premises

1. The introduction of **DENA** should not require the [admins] to provide additional infrastructure.
2. The DENA-APP **retrieves** the [data] **directly** from the [admin] through DENA-CORE **acting as PROXY** (an *intermediator*) **between the [person] who consumes the [data] and the [admin] providing the [data]: NO [data] is stored at DENA-CORE**
3. Every time a [person] retrieves [data] from an [admin] a **[consent]** provided by the [person] is **mandatory**: the [admin] can check in DENA-CORE if the [consent] is valid and legitimate.
4. **DENA-APP stores the retrieved full [data] from the [admins] in a [local cache]** at the [mobile / web app] so it can show / “paint” [data] in the [user interface] as soon as possible from that local cache and refresh it in the background.
5. The DENA-APP must know about:
 - **Data Provenance**: which [data origins] (admins) hold information about a [person], enabling the DENA-APP to determine exactly **where to retrieve [data]**.
 - **Data changes in the [data source]** (admins), allowing the DENA-APP to determine if its locally cached [data] requires a refresh or not.
6. The DENA-APP must have a **fluent behavior**: the [user] should NOT have to “find”/browse in the [app] for the [data] by clicking / navigating [admins], [themes] or anything (a *[web browsing] paradigm*); DENA should show directly and clearly the available [data] for the [person], not requiring the [user] to find it

The full [data] is **NEVER** stored at DENA-CORE BUT it’s locally cached at the DENA-APP in the [user]’s [client device]

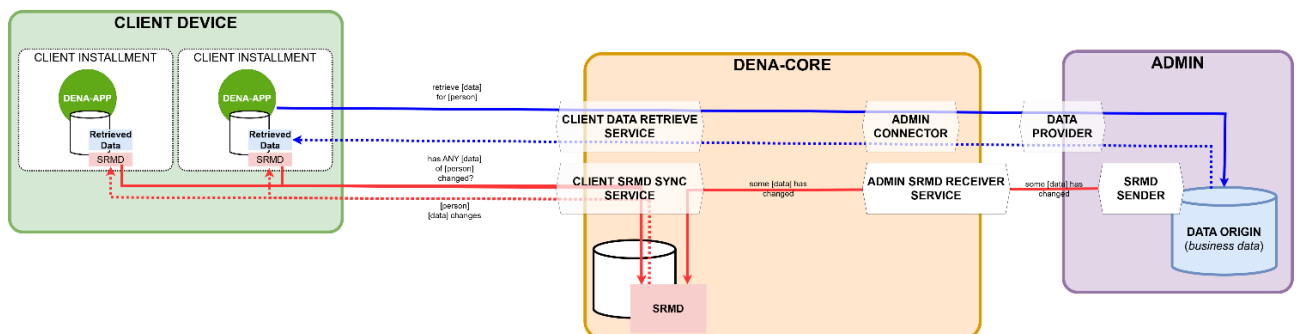
Even though **DENA** never stores the [person] – [admin] interchanged [data], some [meta-data] is needed at DENA-CORE to provide the DENA-APP functionality.

1.2 DENA Concepts

The following are the most important concepts of DENA that is worth knowing to understand DENA documentation:

- **[person]** an individual (*for the moment just physical persons*) that **enrols** in DENA (*creates a DENA account*)
- **[admins]** are **public entities that exposes [data]** to the DENA ecosystem
- **DENA-APP** a [mobile app] or a [web browser app] used by [persons] to access his/her [data] in different [Admins]
- **DENA-CORE** is the system that provides services for:
 - The DENA-APP
 - The interoperability between DENA-APP and the [admins]
- **[client device]**: a mobile phone, tablet, pc or any system in which DENA-APP is installed
- **[client installment]**: a [person] can use DENA-APP (*mobile / web*) in multiple [client devices], for example, a [person] can have many [mobile devices] in which he/she installs the [mobile app] or he/she can open the [web app] in multiple [web browsers]. Each of these [mobile app] *installations* or [web browser] *sessions* are considered a [client installment]
- **[data origin]**: a source of [person]'s [data] in an [admin]
- **[data provider]**: a service exposed by the [admin] that DENA's [admin connector] calls to **retrieve** [data]
- DENA **[admin connector]**: a DENA-CORE component that intermediates between DENA-CORE and the [admin]'s [data provider]
- **[SRMD]** (*Sync And Retrieve MetaData*): meta-data stored at DENA-CORE that provides DENA-APP info about:
 - **Data Provenance**: which [data origins] (admins) hold information about a [person], enabling the DENA-APP to determine exactly **where to retrieve [data] from**.
 - **Data changes in the [data source]** (admins), allowing the DENA-APP to determine if its locally cached [data] requires a refresh or not.
- DENA **[admin SRMD receiver]**: a service exposed by DENA for [admins] that [admins] call to send SRMD
- DENA **[client SRMD SYNC]**: a service exposed by DENA for [client installments] that DENA-APP calls to SYNC its local SRMD copy

The following image summarizes all concepts:



1.3 DENA [data] interchange mechanism high level view

DENA [data] interchange has two main concepts:

- **SYNC [meta-data]** about changes of [data] at the origin [admin]
- **RETRIEVE the full [data]** from the [admin]

1.3.1 SYNC [meta-data] about changes of [data] at the origin [admin]

DENA-APP is based on a [local cache] where all retrieved [data] is stored and used in DENA-APP if there's no indication of a change in the [data source] (*admin*)

For populate and refresh that [local cache], DENA-APP must know about:

- **Data Provenance:** which [data origins] (admins) hold information about a [person], enabling the DENA-APP to determine exactly **where to retrieve [data]**.
- **Data changes in the [data source]** (admins), allowing the DENA-APP to determine if its locally cached [data] requires a refresh or not.

In other words, DENA-APP must have a mean to know **in what [data origins] (*admins*) is there new / updated [data]** and that mechanism is the **[Sync And Retrieve Meta-Data] (SRMD)** which is sent to DENA-CORE by the [admins] **advising about changes in every [person]'s [data]**

The [admins] send DENA-CORE [SRMD meta-data] which is composed of records with the following fields:

```
[person] | [admin] | [data type] | [last update instant]
```

With this information, the [admin] is "telling" DENA-CORE that:

```
"Some [data] of a certain [data type] about a [person] has changed"  
(and when it has changed)
```

This info **DOES NOT tell DENA-CORE what exact [data] has changed**; it just *says* that one or many (*unknown*) [data items] of a [data type] have changed and when was the last (most recent) change

For example: it DOES NOT *tell* DENA-CORE what exact [admin record] (*expediente*) has changed; it just *says* that one or many (*unknown*) [admin records] (*expedientes*) have changed

DENA-CORE knows nothing about the concrete [data item] or the kind of [change]; it just knows that there's a change.

The [SRMD meta-data] is a **very important part of DENA** because it allows the DENA-APP to know:

- If there's new / updated [data] at the [data origin] (*admin*)
- ... and in that case, what [data origin] (*admin*) to call to RETRIEVE the full [data]

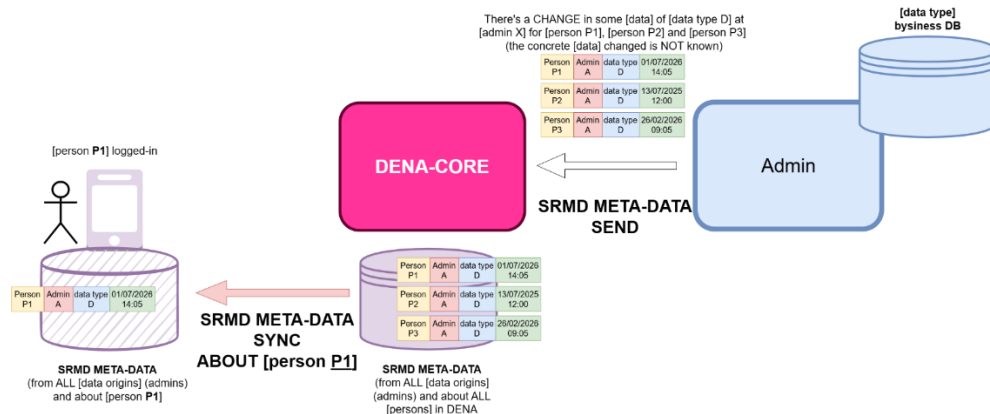
The [SMD meta-data] is sent by the [admin] and synchronized at the DENA-APP, so there are two stages:

1. All [admins] **periodically** in the background **send [SRMD meta-data] to DENA-CORE with changes** in the [data types] about all the [persons] in DENA
2. The **DENA-APP (at the [user]'s [client device]) gets from DENA-CORE the [SRMD meta-data]** for the [person] logged-in at the DENA-APP and **stores it in a local DB** (a cache at the [client app])

DENA-APP can know in what [data origins] (admins + data type) exist changes by comparing the locally-stored [SRMD] with the DENA-CORE copy that was updated by the [SRMD] sent by the [admins].

For example, if the DENA-APP has in its [local cache] that [data type X] has changed at T1 and the received [SRMD] from DENA-CORE says that [data type X] has changed at T2 (*and T2 > T1*) then DENA-APP knows that must update (RETRIEVE) that [data type] (*more on this later*)

The following image resumes the system (look at it from right to left):

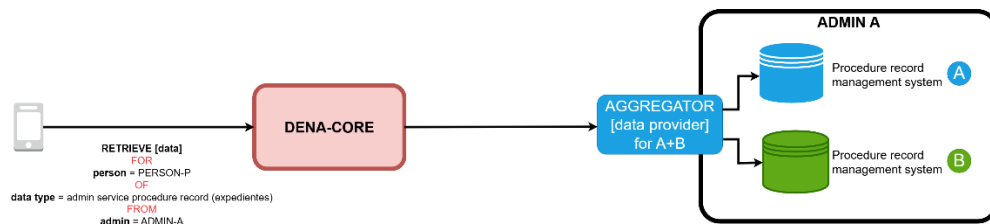


In the previous SYNC mechanism, there is a catch: situations arise where an [admin] hosts multiple [data origins] for the same [data type].

For example, an [admin] might operate **multiple** [procedure record management systems] — *gestores de expedientes* — that **provide data to DENA for the same [data type]=[admin procedure] (expedientes)**.

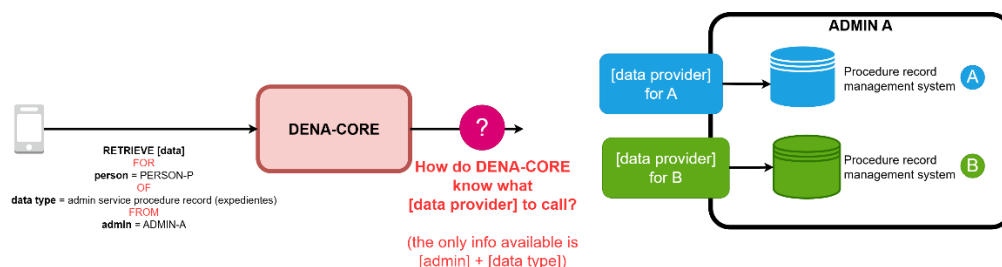
In this scenario, the [admin] can opt for one of **two** architectural patterns:

1. **[aggregated data provider] pattern:** Provide [data] through a **single**, common [provider service] that provides [data] on behalf of multiple [business DBs]



This pattern is straightforward for DENA-CORE, **but it requires the [admin] to build a [data aggregator]** that offers [data] through a single common [data provider] for multiple [backends] (*procedure record management systems*)

2. **[distributed data provider] pattern:** Provide [data] through **multiple** separate [data providers]



While this pattern may be easier for the [admin], **how does DENA-CORE know which [data provider] to call if the only routing information it has is the [admin] and the [data type]?**

The SRMD discussed so far (person | admin | data type | last update instant), supports the [aggregated data provider] pattern, BUT it **DOES NOT** support the [distributed data provider] pattern because DENA-CORE **does not have enough *routing* information to know what [data provider] to call.**

The SRMD record needs another meta-data that provides *routing* information to DENA-CORE to know the [data provider] to be called. That metadata is the [data origin instance]

[person] | [admin] | [data type] [data origin instance] | [last update instant]

The [data origin instance] value is determined by the [admin] and configured in the *routing table* of DENA-CORE to determine what [data provider] to call

- If [admin]=ADMIN A and [data type]=PROCEDURE_RECORD and [data origin instance]=A then CALL [data provider] for A
- If [admin]=ADMIN A and [data type]=PROCEDURE_RECORD and [data origin instance]=B then CALL [data provider] for B

1.3.2 Retrieve the full [data] from the [data origin] (admin)

The previous section explained the mechanism that allows DENA-APP to know what [data origins] have new/updated [data] that must be **retrieved from DENA-APP**

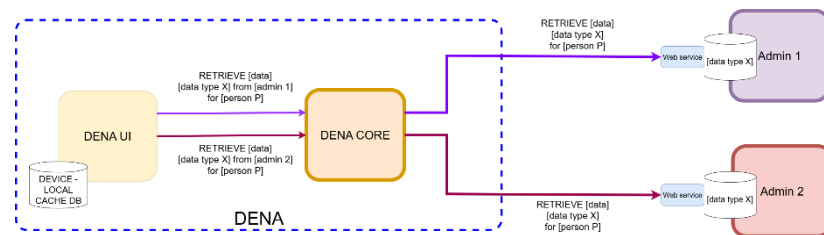
It's important to remember that:

Once the full [data] is retrieved from the [data origin] (*admin*), the [data] is **locally cached at DENA-APP**, so it **does not have to be retrieved again if there's no indication of a [change] in the [data origin] (admin)**

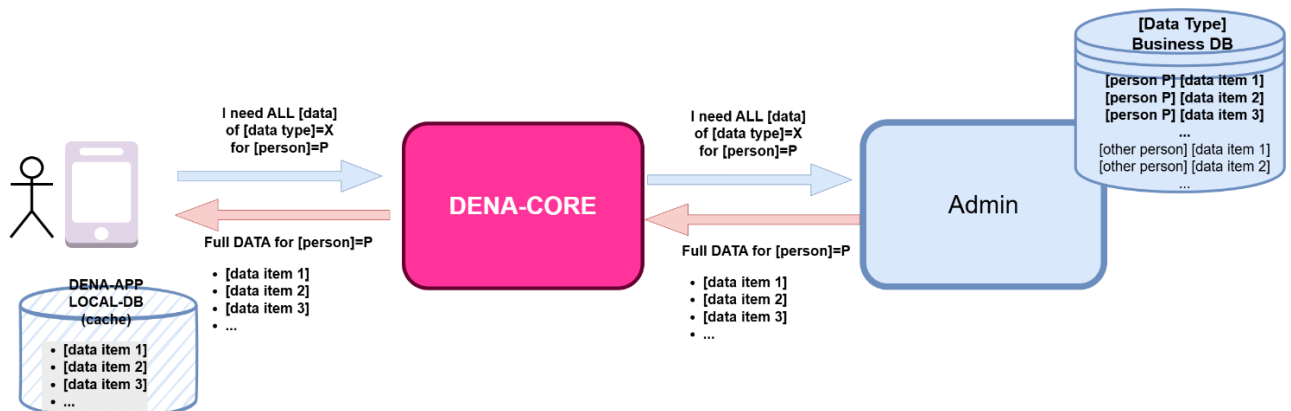
That locally-cached [data] is used by DENA-APP to “paint” [data] to the [user] as soon as possible

This section describes how that **retrieval** process is done.

The [data retrieval] is quite straightforward and just **uses DENA-CORE as a proxy (intermediator)** with the [data origin] (*admin*)



Another view of the previous:



As shown, **DENA-CORE** acts as an intermediary / PROXY for DENA-APP to retrieve the full [data] from the [admin]

This way, DENA-APP is **decoupled** from the [Admins] and do not need to know anything about them; all the [semantic] complexities are handled by DENA-CORE.

... so, DENA-APP always *talk* with DENA-CORE with **standard [semantics]**:

- DENA-CORE exposes a REST-service through HTTPS
- DENA-APP identifies itself at DENA-CORE using OAUTH
- The [data] interchanged for every [data type] adjust to a **standard format** (*each [data type] has it's own standard format that models the interchanged info*)

From the technical point of view:

- DENA-APP (DENA-UI) uses a **common [service] at [DENA-CORE]** where it “asks for” the [data]
- The [admin] must **expose a [data provider] service** that is called from [DENA-CORE] to retrieve the [data] for the DENA-APP

This proxy architecture **decouples** DENA-APP from the multiple [data providers] at the [admins]: DENA-APP just *talks* with DENA-CORE.

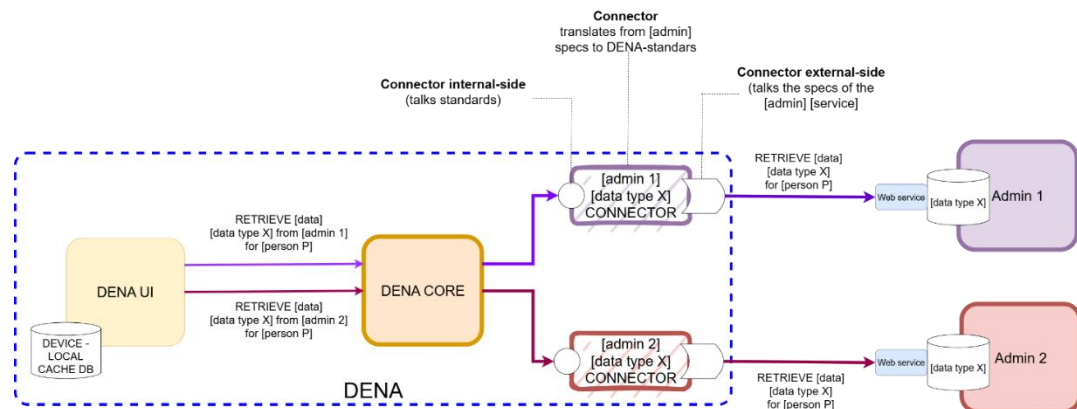
The implementation of the [data provider] at the [admin] that **provides [data] for retrieval** can choose between:

1. Build the [data provider] using the **DENA standard [semantics]**:
 - The [service] is exposed as an HTTP REST Service
 - DENA-CORE authenticates at the [admin] using OAUTH
 - Standard [data format] (*standard semantics*)
2. Build the [data provider] with **CUSTOM [semantics]** in one or many of the areas, for example:
 - [service] exposed with HTTP semantics different from a REST Service (*a SOAP-Web Service*)
 - DENA-CORE authenticates at the [admin] using an X-509
 - The [data format] (*semantics*) returned by the [service] does NOT align with the [standard]

Whatever approach the [admin] chooses (*standard impl or custom impl*), DENA-CORE needs to “something” that **translates** between:

- The [standard semantics] used internally in DENA-CORE and in DENA-APP
- The [semantics] (*custom or standard*) used at the [data provider] at the [admin]

DENA-CORE uses a [**connector**] architecture like the one depicted below:



The function of a [connector] is the **translation of the DENA [semantics] used by DENA-CORE to the [semantics] used by the concrete [data provider]** for [data] of a certain [data type] in an [admin]

A [connector] has two sides:

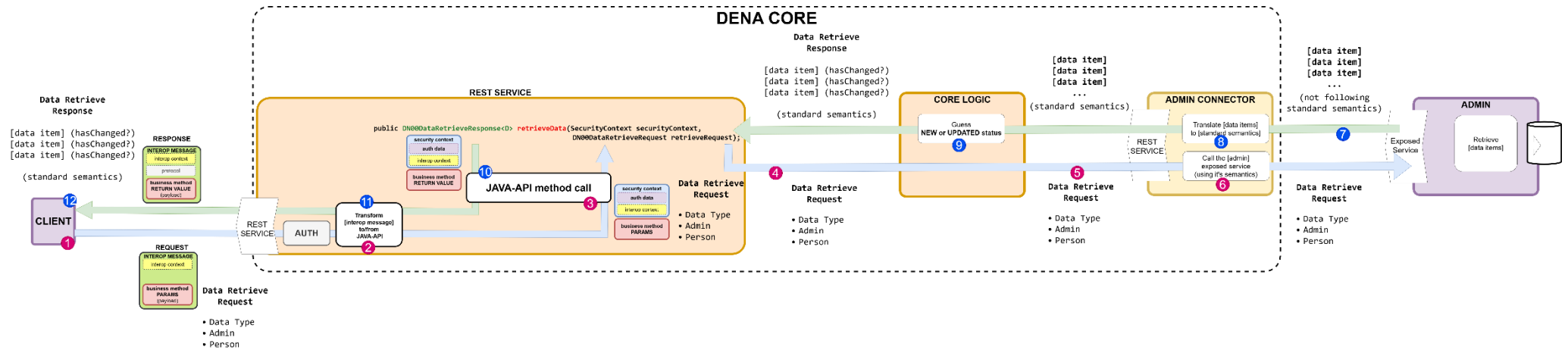
- The [**internal side**] that is used by the DENA-CORE using **DENA-standard [semantics]** (*transport, security and data format*)
- The [**external side**] that knows how to interact with the remote [data provider] exposed by the [admin] in terms of:
 - **Transport:** *how to connect to the [remote service], for example the http(s)-specific details (url, urlpath, headers, etc)*
 - **Security:** *how DENA-CORE identifies itself at the [remote service], for example using an OAUTH service provided by the [admin] or a X509 certificate*
 - **Data Format:** *the interchanged [data] format*

The [connector] **translates** the [semantics] of both sides: the standard one used internally and the [admin]-specific one used externally

... anyway, an [admin] can expose a [data provider] that use **standard [semantics]** so the [connector] just acts as a transparent proxy from DENA-CORE to the [data provider] remote service.

[connectors] are **independent modules** that are deployed independently of DENA-CORE to make the whole system more **resilient**: if an [admin] [data provider] throughput starts to degrade and returns the response very slowly, connections from the [connector] can stuck and cause problems locally at the [connector], but never affect to anything else in DENA system.

The following figure resumes all the steps for a [data retrieve] request:



- [1] - The client calls a DENA-CORE REST-Service requesting the [data retrieval] sending an [interop message] that contains the [data type], [admin] and [person] whose data is required
- [2] - The REST-Service authenticates DENA-APP and **translates** the [interop message] to [model objects] used by the JAVA-API
- [3] - The JAVA-API **retrieve data** method is called
- [4] - The JAVA-API calls the CORE LOGIC
- [5] - The CORE LOGIC guess what [admin connector] has to be used and calls the REST-SERVICE exposed (*which talks standard semantics*)
- [6] - The [admin connector] **calls a [service] exposed by the [admin]** that provides the [data tiems]
- [7] - The [data provider] service exposed by the [admin] **retrieves de [data items]** from the [business DB] and returns them to the [admin connector] (*note that the returned data DO NOT HAVE to follow the [standard semantics]*)
- [8] - The [admin connector] -if necessary- **translates the returned [data items]** from an [admin]-custom [semantics] to the [standard semantics]
- [9] - The CORE LOGIC guess the “**has changed**” state of every [data item]
- [10] - The JAVA-API returns the [data items] (*in standard semantics*) with the “**has changed**” status
- [11] - The REST-Service **translates** the JAVA-API returned [data items] to a **response [interop message]**
- [12] - The response [interop message] arrives to the [client]

It's important to know **how DENA-CORE guess the [has changed] status of every [data item] returned by the [admin]** used by DENA-APP to **highlight the NEW or UPDATED [data items]** in the UI

The [admin] does NOT need to send if a [data item] is NEW or has been UPDATED because **DENA-CORE guess that state while [data items] passes through DENA-CORE during [data retrieval]** (*remember that **DENA-CORE does not store [data items]***)

The logic that DENA-CORE uses is:

1. Every [data item] returned by the [admin] has a **lastChangedAt** field with the last instant when that [data item] has changed at the [admin]
2. DENA-CORE loads the **last time the [person] issued a [data retrieval]** (*before this one*)
3. If the last time the [person] issued a [data retrieval] is **BEFORE** the [lastChangedAt] instant of the [data item], then the [data item] is NEW or has been UPDATED

The most important info that the [admin] must include in each [data item] is the **last time the [data item] has changed** (lastChangedAt)

NOTE that the {last time the [person] issued a [data retrieval]} is **loaded for the [person] and NOT for the [client installment]**; this way the behaviour is consistent across [client installments]

1.3.3 The COLD-START problem

When a [person] enrolls in DENA, no SRMD exists at DENA-CORE because **the [admins] DO NOT send SRMD for [persons] who are not yet enrolled.**

Consequently, DENA-APP will NOT display anything until:

1. The [admins] know about the [person] (the [person] is synced to the [admins]).
2. The [admins] send the SRMD to DENA-CORE.
3. DENA-APP syncs the updated SRMD

This process can take a while (from minutes to hours), so **DENA-APP should display some [data] as soon as possible** to provide a basic user experience.

To achieve this, a trick is used and **forces DENA-APP to retrieve [data] from certain [admins]/[data types]**

1. **Immediately after [person enrolment]**, DENA-CORE inserts several SRMD entries as if the [admins] had sent them for key [admins]/[data types] across high-level [admins] (e.g., EIGV, DFB, DFG, DFA, Bilbao, Gasteiz, Donostia)
2. When DENA-APP syncs SRMD, it will recognize that it must **retrieve** [data] for those key [admins]/[data types].

While the [person] may not have [data] across all [admins], this approach maximizes the chances that at least one or more [data origin] returns [data] to display in DENA-APP.

When a [person] starts using DENA (*the first time he/she enters DENA*), no [admin] have sent any [SRMD] meta-data (sync meta-data) so DENA do NOT know anything about the [person] and the [data] she/he has in any [admin], and the [client app] cannot show anything

This is only for the **first [client installment SRMD SYNC]** operations; as soon as the [admins] start sending SRMD [data] to DENA-CORE, the system start to operate as expected.

1.4 Person SYNC

Every [person] that enrolls in DENA will have an **account in DENA** that will store:

- Person OID: *a DENA-generated unique identifier*
- Person ID: *the NIF*
- Full name: name, surname1, surname2
- Contact Data: address, phone, email...
- Preferred themes (*can be used in proactive services or to customize the UI experience*)_

This *simple* [data] collected for every [person] enrolled in DENA is **available to every [admin] participating in DENA**, so the system offers the [admin] means to sync [person] [data].

One of the purposes of **synchronizing [persons] with the [admins]** is to **make them aware of which [persons] are enrolled in DENA**. This ensures the [admin] sends only the **[SRMD meta-data]** for those [persons], and not for anyone who does not have a DENA **[account]**

DENA provides the following means to the [admins] to sync [person] data:

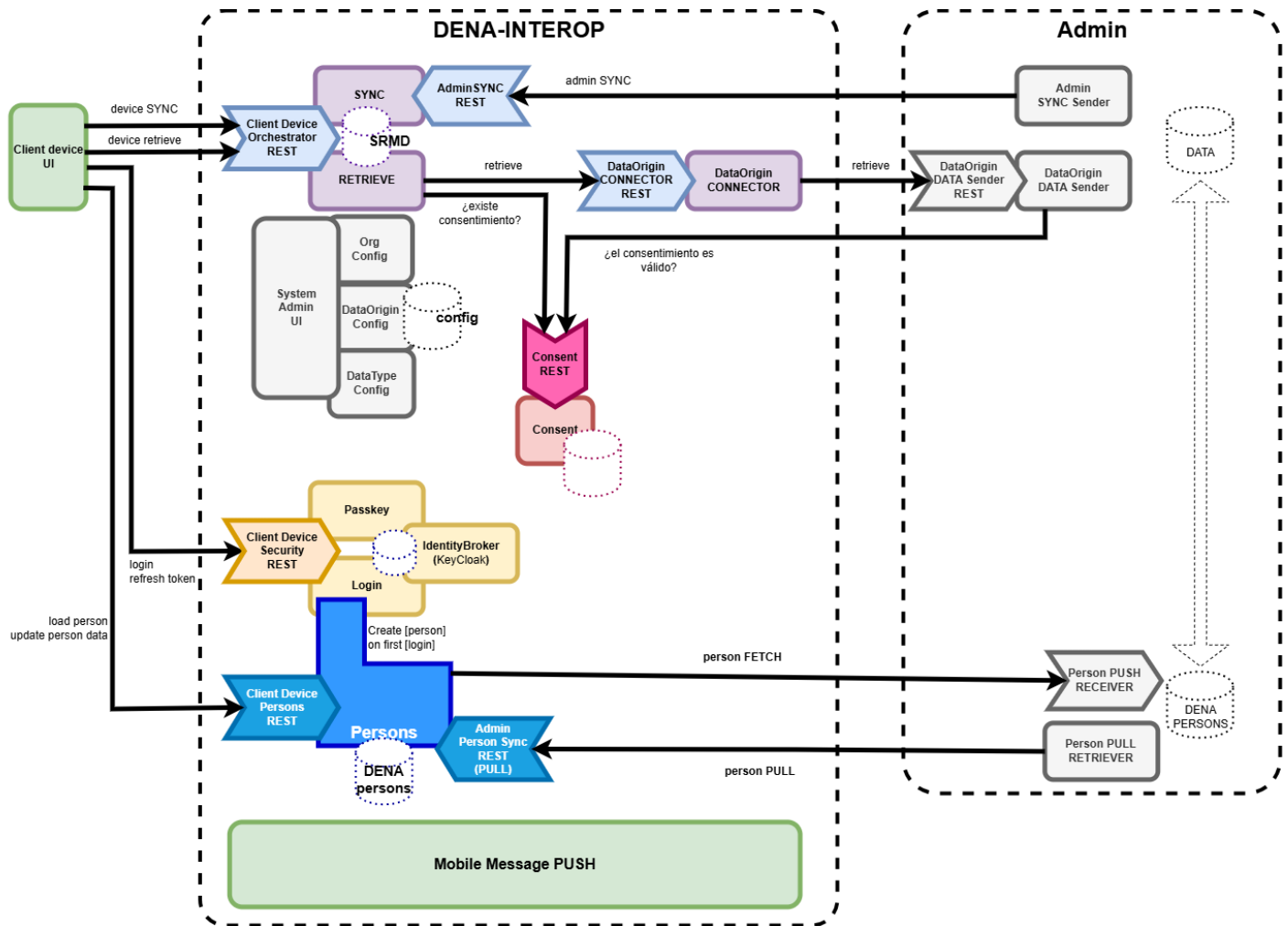
- **DENA-PUSH:** DENA sends a message to a [service] in the [admin] whenever there's a change in a [person] in DENA
 - *a new [person] is enrolled DENA*
 - *a [person] deletes the [account]*
 - *a [person] changes some of the basic data (full name, contact data, etc)*
- **ADMIN PULL:** the [admin] retrieves the [person] data from DENA
 - On-line: DENA publishes a [REST service] where any identified [admin] can query [person] data
 - Individual [person data]: invoke the REST service using a [person id] (NIF) to get that [person]'s [data]
 - Multiple [person data]: invoke the REST service using a [person query] to get a **paginated [search result]** with the query-matching [person]'s [data]
 - Off-line (*batch for multiple*):
 - Pre-generated: DENA pre-generated files with [person] data
 - Bespoke: the [admin] requests a custom [person] data file that is generated by DENA for the [admin]

BEWARE that this [person sync] should not be confused with the [contact data] [data type]:

- The [person sync] is a means for the [admins] to have access to all the [person] account data in DENA
- The [contact data] [data type] is just another interoperable [data type] (similar to [admin procedure records] / *expedientes*) that allows a [person] to view their own [contact data] across multiple [admin] databases that store it.

2 DETAILED ARCHIECTURE

The following is a detailed diagram of the DENA architecture:



The main blocks of the architecture are:

- [client device UI]: **DENA-APP**
- DENA-CORE composed by:
 - [config] module that contains parameters used by the system:
 - **Organizational structure:** [admin groups] and [admins]
 - **Interop config:**
 - [Data types] offered by the [admins]
 - [Data origins] config used by the [connectors] to know how connect to [admins] in the **retrieve** phase
 - [App Version] history
 - An [admin UI] to manage all the [config] objects
 - [persons] module responsible for:
 - The DENA [person account] which stores some data as ID, full name or contact data
 - The [person data SYNC] to the [admins] that can have a copy of the DENA [person] DB

- **[sync and retrieve]** module (SRMD) provide services for:
 - [Admins] that **send** DENA-CORE **changes** y [data] about [persons] (SRMD)
 - DENA-APP that:
 - **uses** the SRMD to guess if [data] should be **retrieved** from the [data origin] (admin) or not, in which case, the DENA-APP-locally-cached [data] can be re-used
 - **retrieves** [data] from the [data origin] (admin) using **[connectors]** that **translates** DENA standard [semantics] to the [data origin]-specific [semantics]

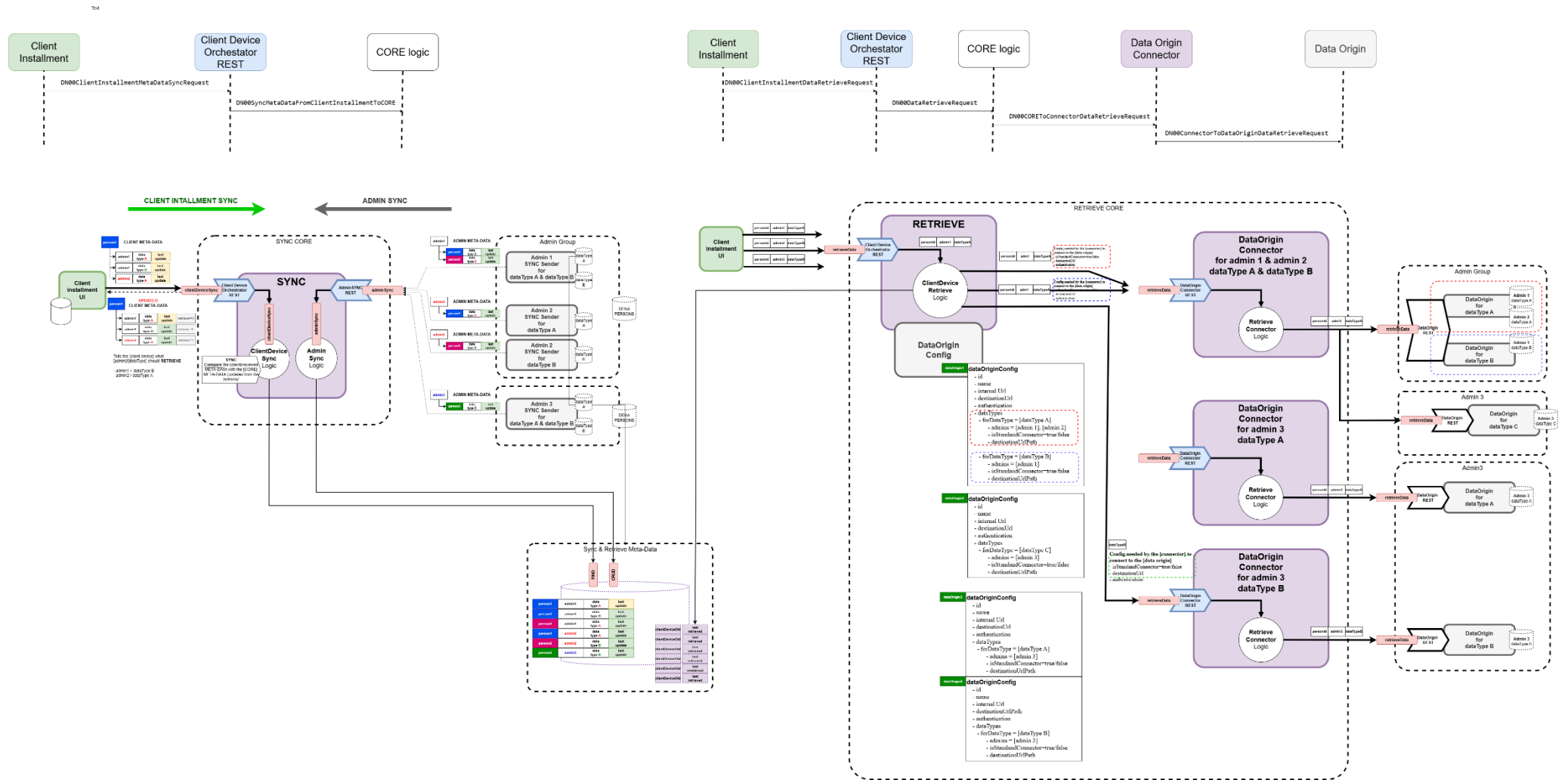
- **[consents]** module that:
 - **Stores** the [contents] given by [persons] about the [data] usage
 - **Checks** that every [data] usage is backed by a valid [consent]

- **[security]** module in charge of core logic of:
 - The [person] **login** at the DENA-APP
 - The first time the [user] log-in using [gilza]
 - ... and once the [user] has logged-in [passkeys] are used to avoid requiring using [giltza] again
 - The [admins] auth when sending SRMD [data]
 - The [users] auth on the [admin UI]

2.1 Sync and Retrieve

The following is a detailed view of the [sync and retrieve] modules

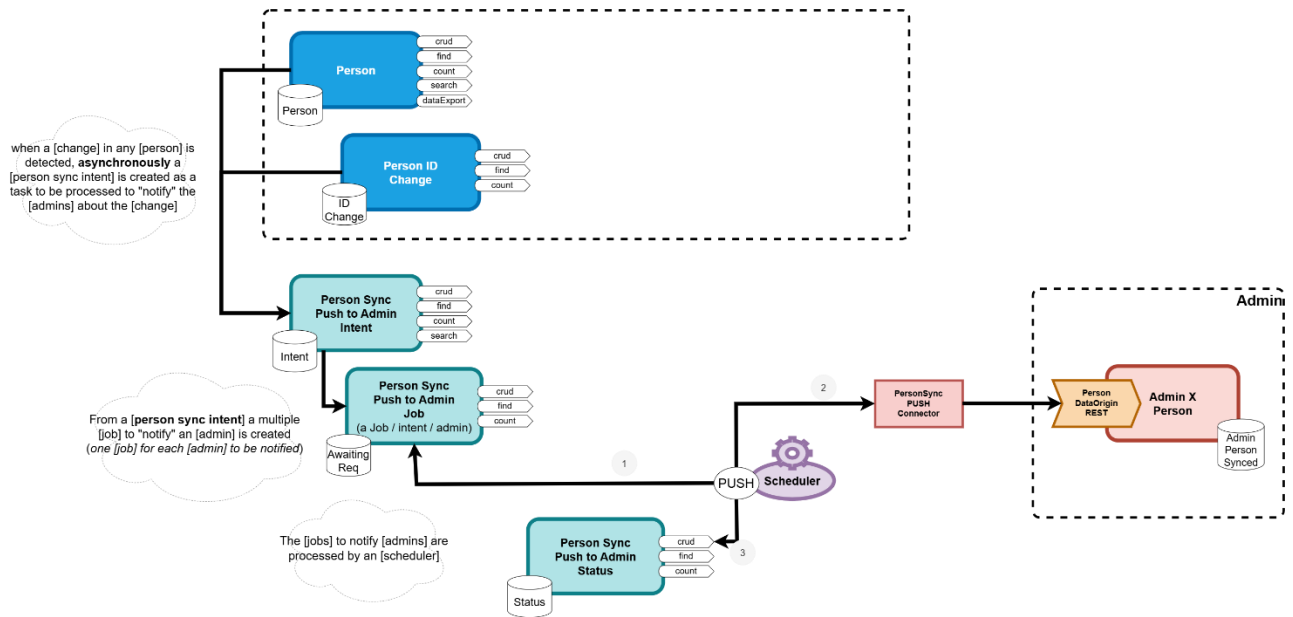
- The left part depicts both [SRMD SYNC] parts:
 - [Admin] SYNC: how the [admin] sends SRMD to DENA-CORE
 - [client installment] SYNC: how the DENA-APP gets SRMD from DENA-CORE
- The right part depicts the [client installment] [data RETRIEVAL] and how the [data origin] config is used



2.2 Person Sync

2.2.1 DENA Push Sync

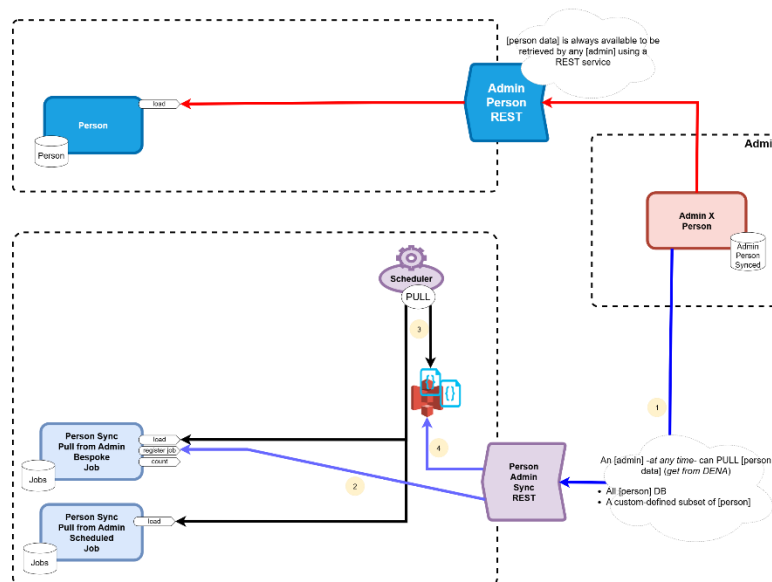
The [person PUSH sync] detailed architecture is like (read it from left to right):



- When a [change] in any [person] is detected, asynchronously a [person sync intent] is created as a [task] to “notify” the [admins] about the [change]
- Since there exist multiple [admins] to be notified, the [person sync intent] is converted to multiple [notify jobs] (*one for each [admin]*)
- Each [notify job] is processed by an [scheduler] one by one which **sends the [admin] all the [person data] (updated)**

2.2.2 Admin Pull Sync

The [person PULL sync] detailed architecture is like (read it from right to left):



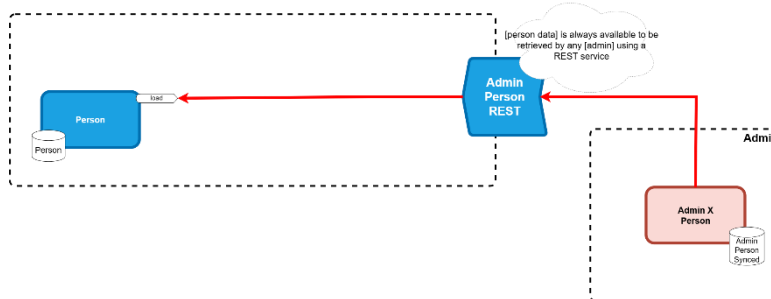
There are three options for the [admin] to pull [person data]:

- Online: pull [person data] using a REST-Service
- Batch:
 - Pre-generated files with multiple [person data]
 - Bespoke (*custom*) file with multiple [person data]

2.2.2.1 Online admin-pull of [person data]

DENA offers a REST service that can be used by an [admin] to get:

- Individual [person data]: invoke the REST service using a [person id] (NIF) to get that [person]'s [data]
- Multiple [person data]: invoke the REST service using a [person query] to get a **paginated [search result]** with the query-matching [person]'s [data]



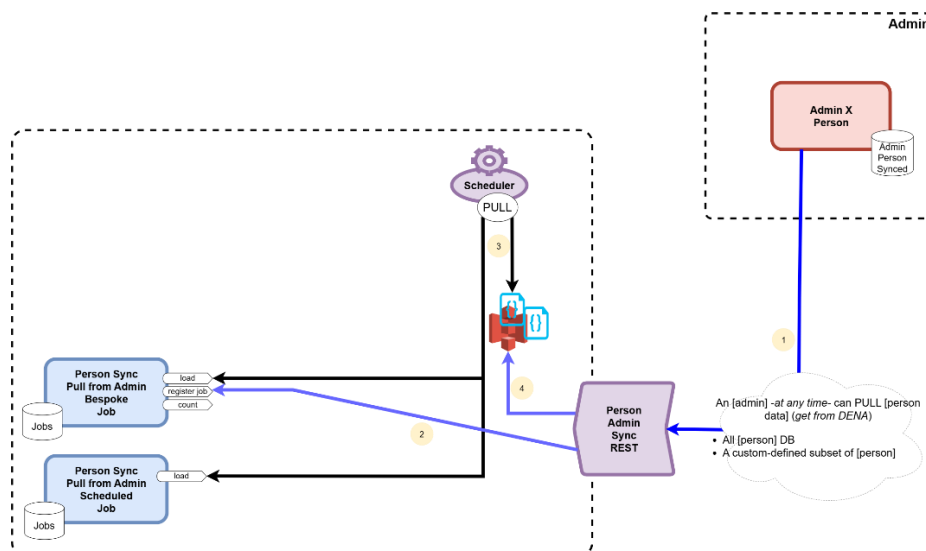
2.2.2.2 Offline (batch) admin-pull of [person data]

If an [admin] needs multiple [person data], the online approach using query to get paginated search results is not always the best solution, especially if there's a lot of records to be pulled

As an alternative, DENA offers an offline (batch) mechanism that allows an [admin] to **pull a file with the records**.

There are two options:

- Pull a DENA-pregenerated file: DENA periodically pre-generates some files with [person data] for retrieval by the [admin]
 - A file with all the [person] with a DENA account
 - A file with all the [person] whose [data] has been changed in the last X time (e.g. *past day*, *past hour*...)
- Pull an [admin]-defined (*bespoke*) file: the [admin] requests DENA to generate a file containing [person data] that matches [admin]-defined criteria



In each of these approaches, the [admin] simply calls a DENA REST service that returns the required file (either pre-generated or bespoke):

- **Pre-generated file:** The REST service returns the file directly in the response
- **Bespoke file:** The [admin] submits the filter criteria, and the REST service returns a [job token]. The [admin] then polls this token periodically to check if the file is ready for download

3 SECURITY

Security in DENA has four objectives:

1. **Legitimate request source:** Ensure that the [service] is accessed only by authorized and verified parties.
2. **Request integrity:** Ensure that interchanged data is not altered end-to-end during transit.
3. **Data confidentiality:** Ensure that interchanged data remains encrypted and invisible to unauthorized third parties.
4. **Prevent request replay attacks:** Prevent malicious third parties from capturing and replaying requests.

To achieve these objectives multiple security mechanism are implemented:

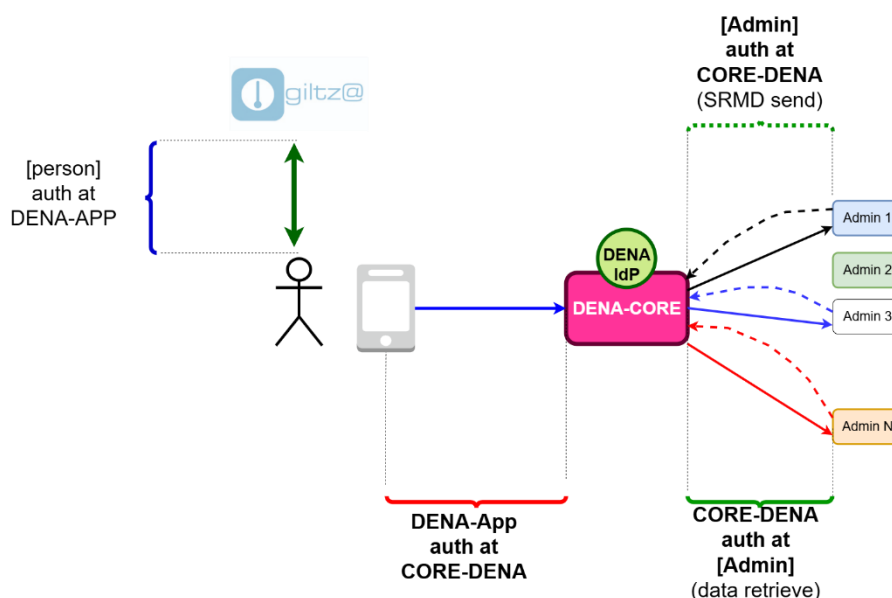
	HTTPS	JWT	Security Header
Legitimate request source		X	X
Request integrity		X	X
Data confidentiality	X	X	
Prevent request replay attacks		X	X

There are TWO different concepts:

- **Authentication / identification:** know **WHO** the third party is and **provide** him with **means to use the system** (a [security token], a [security session] or whatever mean)
- **Authorization:** **WHAT** the [authenticated] party **can do** in the system

Here only the [authentication] is described

There are four areas in which different parties are [authenticated] as depicted below:



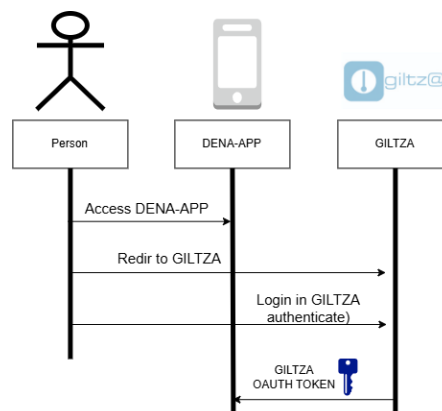
- **[person] auth at DENA-CORE:** The [person] authenticates with DENA-APP using a GILTZA token (OAUTH)
- **DENA-APP auth at DENA-CORE:** DENA-APP authenticates with DENA-CORE using a DENA-IdP token (OAUTH)

- **[Admin] auth at DENA-CORE:** The [Admin] authenticates with DENA-CORE using a DENA-IdP token (OAUTH)
- **DENA-CORE auth at [admins]:** DENA-CORE authenticates with [Admins] he target [admin]'s preferred [auth system] (*e.g., OAuth, X.509 certificates, username/password*)

3.1 [person] auth with DENA-APP: GILTZA OAUTH token

The flow is quite simple:

1. The [user] (person) enters DENA-APP and he/she is **redirected to GILTZA login page** using a web-view (web browser)
2. The [user] (person) log-in
3. GILTZA returns an [oauth token]

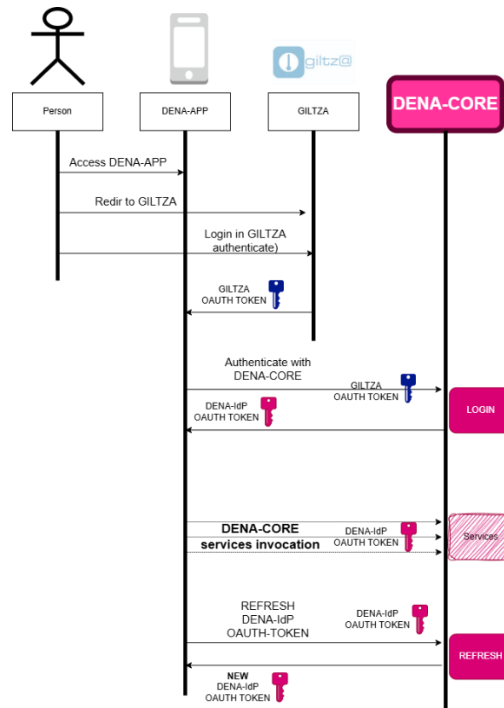


3.2 DENA-APP auth with DENA-CORE: DENA-IdP OAUTH token

DENA-APP identifies itself with DENA-CORE using the previously provided [GILTZA-oauth token]

In exchange, DENA-CORE returns a [DENA-IdP oauth token] that will be sent later by DENA-APP in every DENA-CORE service invocation for example for:

- Init DENA-APP
- Sync SRMD
- Retrieve Data
- etc



BEWARE that the DENA-IdP [oauth token] has a limited duration (**the DENA-IdP [oauth token] expires**) so DENA-APP must **refresh it** periodically

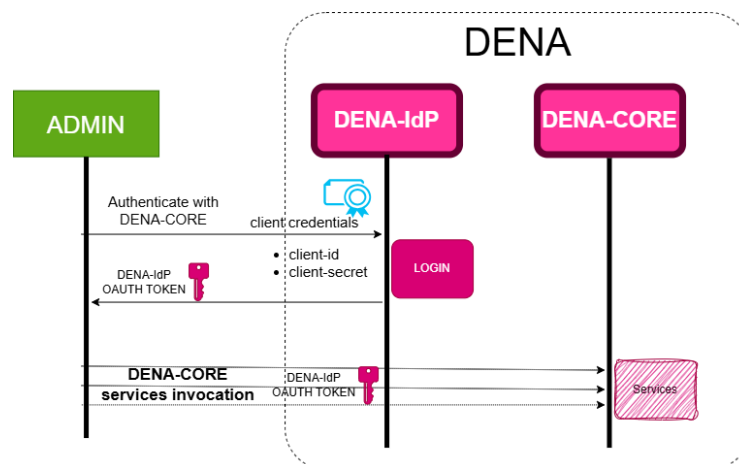
3.3 [Admin] auth with DENA-CORE: DENA-IdP OAUTH token

The [admin] authenticates in DENA sending DENA-IdP a previously provided [client credentials] (*client-id + client-secret*)

The [admin] must ask DENA administrators for the [client credentials]

In exchange, DENA-IdP returns a [DENA-IdP oauth token] that will be sent later by the [admin] in every DENA-CORE service invocation for example for:

- Send SRMD meta-data
- Get [person] details
- etc

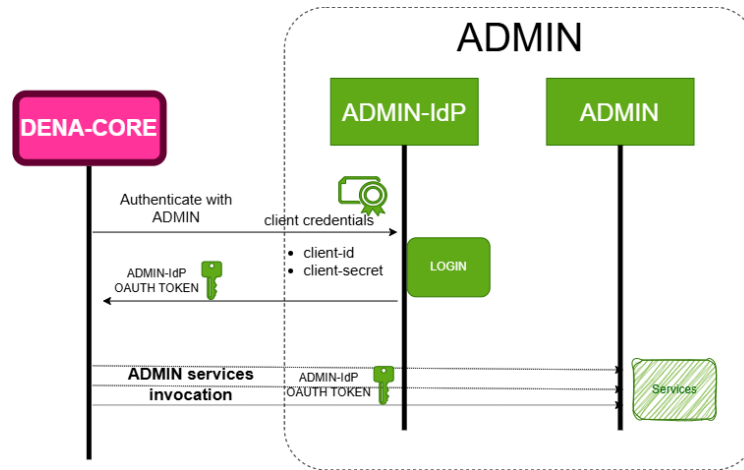


3.4 DENA-CORE auth with [Admin]: target [admin] preferred [auth] system (OAUTH, X-509 certificate, user/passwd...)

DENA does NOT standardize / enforce an authentication system at the [admin]-side; each [admin] may use their preferred method, such as:

- OAUTH
- X-509 certificates
- User/password
- Etc

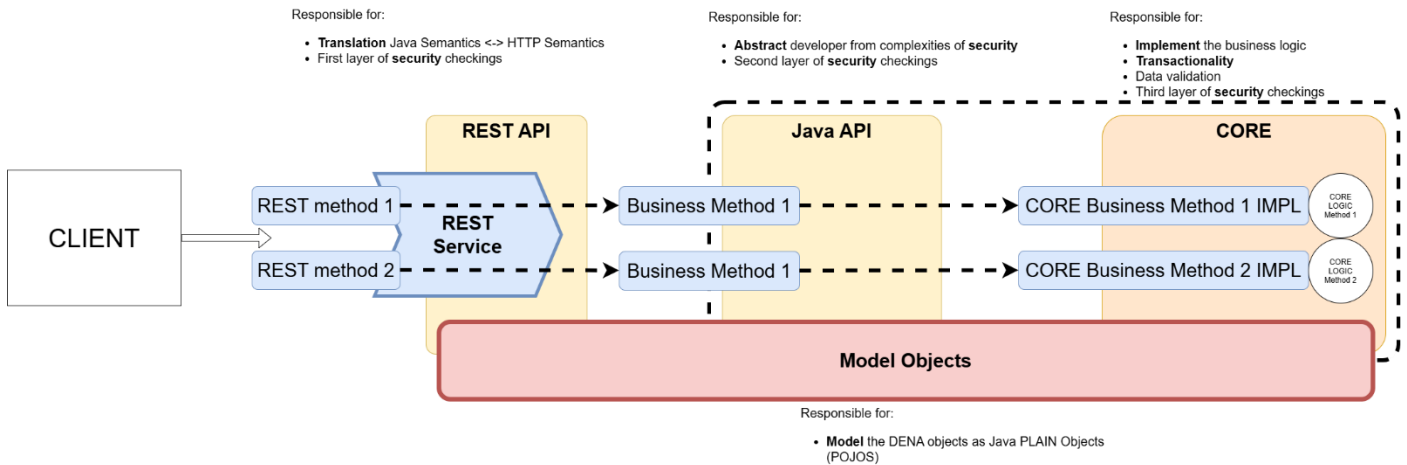
For example, if an [admin] uses OAUTH, the [admin] will provide DENA administrators with **[client credentials]** (*client-id + client-secret*) to use the [admin IdP] and get an [admin IdP OAUTH token] that will be needed to call [admin]-provided services such as **[data retrieval] service**



4 SERVICES INTERNAL ARCHITECTURE

4.1 Overall view

DENA services architecture is depicted below:



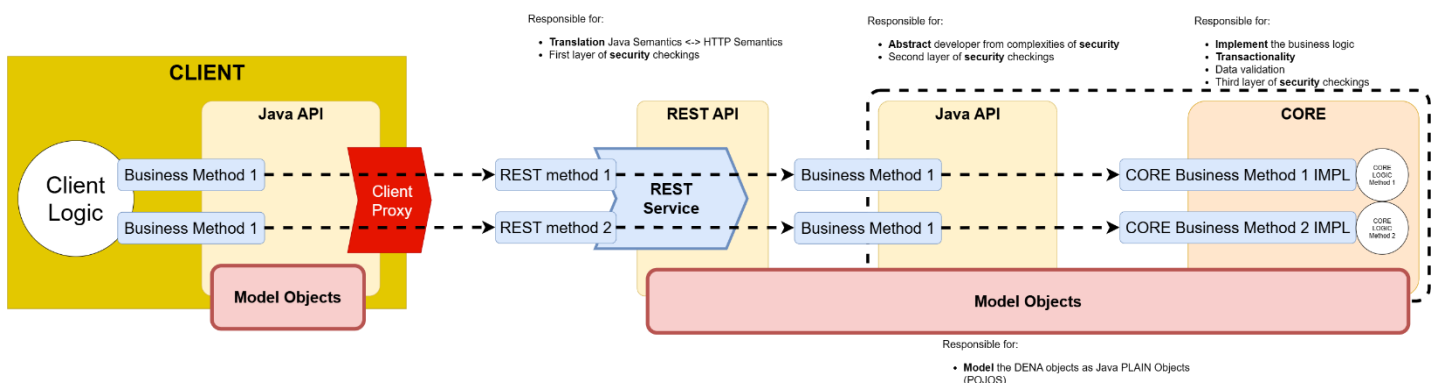
DENA has **two** API levels:

- REST API which exposes CORE services as HTTP methods
- Java API which exposes CORE services as Java methods

The [client] typically *consumes* the REST API, but this is quite difficult since the [client] must deal with:

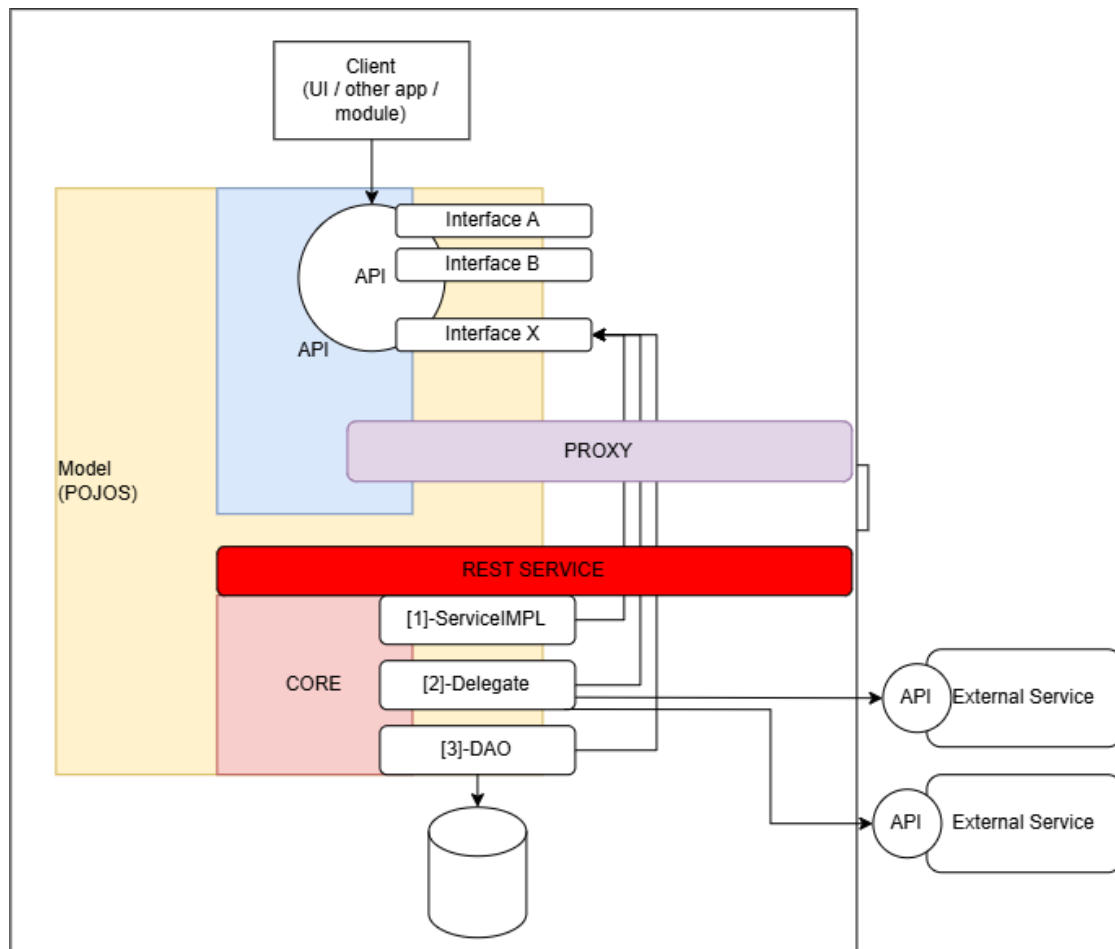
- The HTTP semantics: connections, headers, auth...
- Marshalling (*serializing / deserializing*) the [model objects] to / from JSON to whatever the [client] uses (*java for example*)

So, if the [client] can use the [Java API] the architecture can be like this:



... now the [client] (*using java*) can use the same Java API that DENA-CORE exposes. A [client proxy] handles all the http semantics and data marshalling

An alternative view of the [architecture] with more detail in the CORE implementation is:



The [CORE] typically is implemented in three layers:

1. **[service impl]**: just controls **transactionality**
2. **[delegate]**: param checking and [service implementation] orchestration (*being the most usual one to orchestrate the [DAO] layer*)
3. **[DAO]**: Data access

One of the most important characteristics of the [architecture] is that **every layer implements the [client API interfaces]** ensuring that **everything stays consistent**.

4.2 Interchanged data structures

4.2.1 [interop messages] used at REST-Services vs [model objects] used at JAVA-APIs

The previous section describes the [architecture] from an abstract point of view, but it's important to point out that the **REST-APIs have a [data-semantics] slightly different from the JAVA-API ones**

- JAVA-APIs uses [model objects]
- REST-APIs interchanges [interop messages] that have the following fields:
 - The [payload]: something to be processed
 - The DENA [interop context]
 - [interop message] **origin**: DENA-APP, DENA-CORE or an [admin]
 - [interop message] **type** (ie: *CLIENT_INIT_REQ*)
 - [interop message] **correlation ID** used to pinpoint the call flow
 - [interop message] **route data**: a list of DENA [components] that the [messages] go through
 - Some [protocol] data (if necessary) as:
 - URLs (ie callback urls)
 - Timeouts
 - ...

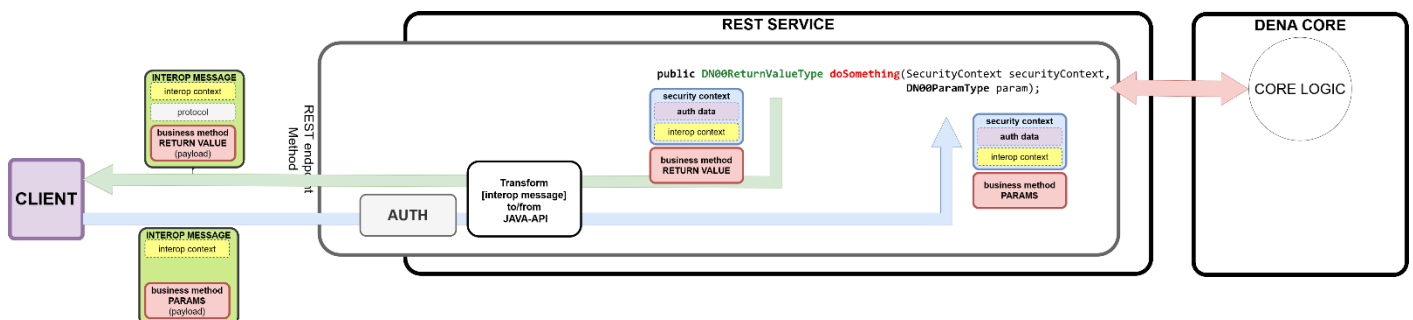
The **important** part of the [interop message] is the [payload]

The [interop context] is used for **debugging / audit** purposes and is NOT strictly necessary

When the [interop message] passes through the REST-API, it's transformed to the [model objects] that JAVA API uses (no JAVA-API methods use the [interop message]s)

- The [interop message]'s [payload] becomes the [business method] **params** or **return value**
- The [interop message]'s [interop context] is also embedded in the [security context] that is always the first param of all the JAVA-API methods

This image resumes this transformation between [interop messages] interchanged in REST Services and [model objects] used at JAVA APIs (read it from left to right starting from the bottom request)



DENA-APP exchange [interop messages] with the DENA-CORE REST-Service which is in charge of:

1. **Authenticate** the DENA-APP
2. **Translate** the received **request [interop message]** into a JAVA-API method **params**
3. **Invoke the JAVA-API method** which in turn executes the CORE-logic
4. **Wrap back the JAVA-API return value** as **response [interop message]** to DENA-APP

For example the **[data retrieval]** for a **[person]** from DENA-APP of **[data]** from fictional **[citizen service appointments]** is like:

1. The **[client installment]** (DENA-APP) **sends** the following **[interop message]** to DENA-CORE composed of the **[interop context]** and the **[payload]**

```
{
  "context" : {
    "message" : {
      "type" : "CLIENT_RETRIEVE_REQ",
      "correlationId" : "1AB7F413-399D-49F1-AC41-44D651E5799A",
      "interopRouteData" : [ {
        "denaComponentId" : "CLIENT_INSTALLMENT",
        "timestamp" : "2026-08-18T11:28:47.5237003Z"
      } ]
    },
    "originClientInstallment" : "8B5AE78A-7D42-4069-A626-959BB07276C5",
    "destinationAdmin" : {
      "id" : "admin-id",
      "oid" : "AFF5E893-2A7E-43C7-BB2B-1E7D1535306D"
    },
    "subjectPerson" : {
      "id" : "personid",
      "oid" : "D530141A-1E2A-4800-A118-3FC8D6EFE6D5"
    },
    "dataType" : {
      "id" : "data-type-id",
      "oid" : "2F92F808-6D5C-4A0F-AB68-3D4D03028CFE"
    }
  },
  "payload" : {
    "dataType" : {
      "id" : "citizen_service_appointments",
      "oid" : "BEDCB4AF-D384-4E05-B74E-A25D7322EF63"
    },
    "dataOriginInstanceId" : "DEFAULT",
    "admin" : {
      "id" : "admin-id",
      "oid" : "D4348C40-84B8-4420-9747-193C75CB2875"
    },
    "person" : {
      "id" : "person-id",
      "oid" : "DAA35E71-5B28-44BF-9DAE-A412E1CEC538"
    },
    "clientInstallment" : "ED4576E0-DF47-4D2F-B039-A91228B3F09E"
  }
}
```

2. The REST-Service logic just **translates** the **[interop message]**'s **[payload]** into the **[model object]**s used by the JAVA-API method **params**

```
public <D extends DN00IsDENADDataExchangedObject>
    COREServiceMethodExecResult<DN00DataRetrieveResponse<D>> retrieveData(SecurityContext securityContext,
                                                                           DN00DataRetrieveRequest retrieveRequest);
```

In this example, the **[interop message]**'s **[payload]** is transformed to a **DN00DataRetrieveRequest** **[model object]**

3. **The JAVA-API calls the CORE-logic** which uses an **[admin connector]** to get to the **[admin]** and ask for the **[data]**
4. The JAVA-API returns a **[model object]**, in this case a **DN00DataRetrieveResponse<DN00ScheduleItem>** because **[citizen service appointments]** (**DN00ScheduleItem**) are being retrieved by DENA-APP.

```
public <D extends DN00IsDENADDataExchangedObject>
    COREServiceMethodExecResult<DN00DataRetrieveResponse<D>> retrieveData(SecurityContext securityContext,
                                                                           DN00DataRetrieveRequest retrieveRequest);
```

5. This **[model object]** (data items) **returned** by the JAVA-API method call is also wrapped back to an **[interop message]** at the REST-SERVICE:

```
{
  "context" : {
    "message" : {
      "type" : "CLIENT_RETRIEVE_RESP",
      "correlationId" : "1AB7F413-399D-49F1-AC41-44D651E5799A",
      "interopRouteData" : [
        {
          "denaComponentId" : "CLIENT_INSTALLMENT",
          "timestamp" : "2026-08-18T11:28:47.5237003Z"
        }
      ]
    }
  }
}
```

```

    },
    {
      "denaComponentId" : "DENA_CORE",
      "timestamp" : "2026-08-18T11:28:47.6217025Z"
    },
    {
      "denaComponentId" : "DENA_ADMIN_CONNECTOR",
      "timestamp" : "2026-08-18T11:28:47.7315121Z"
    },
    {
      "denaComponentId" : "ADMIN",
      "timestamp" : "2026-08-18T11:28:47.7537244Z"
    },
    {
      "denaComponentId" : "DENA_ADMIN_CONNECTOR",
      "timestamp" : "2026-08-18T11:28:47.7722214Z"
    },
    {
      "denaComponentId" : "DENA_CORE",
      "timestamp" : "2026-08-18T11:28:47.8012511Z"
    }
  ]
},
"originAdmin" : {
  "id" : "admin-id",
  "oid" : "AFF5E893-2A7E-43C7-BB2B-1E7D1535306D"
},
"subjectPerson" : {
  "id" : "personid",
  "oid" : "D530141A-1E2A-4800-A118-3FC8D6EFE6D5"
},
"dataType" : {
  "id" : "data-type-id",
  "oid" : "2F92F808-6D5C-4A0F-AB68-3D4D03028CFE"
}
},
"payload" : {
  "requestedNumberOfItems" : 0,
  "itemsPagingContext" : {
    "totalItemsCount" : 200,
    "startPosition" : 0,
  },
},
"dataItems" : [
  {
    "data" : {
      "type" : "scheduleItem", // id used to handle polymorphic types (all [data] types implement DN00IsDENADDataExchangedObject
      "id" : "appointment123",
      "lastChangedAt" : "2026-08-19T08:07:56.7423565Z", // very important!!!
      "aboutPerson" : {
        "oid" : "D530141A-1E2A-4800-A118-3FC8D6EFE6D5"
      },
      "year" : "2026",
      "monthOfYear" : "8",
      "dayOfMonth" : "18",
      "hourOfDay" : "13",
      "minuteOfHour" : "2",
      "durationMinutes" : 30,
      "subject" : {
        "ENGLISH" : "an appointment",
        "SPANISH" : "una cita"
      },
    },
    "urls" : [ {
      "id" : "main",
      "lang" : "BASQUE",
      "value" : "https://my-admin.eus/citizen-service-appointments/appointment123?lang=eu"
    },
    {
      "id" : "main",
      "lang" : "SPANISH",
      "value" : "https://my-admin.eus/citizen-service-appointments/appointment123?lang=es"
    } ]
  },
  "isNewOrUpdated" : false
},
{
  "data" : {
    "type" : "scheduleItem", // id used to handle polymorphic types (all [data] types implement DN00IsDENADDataExchangedObject
    "id" : "appointment345",
    "lastChangedAt" : "2026-08-19T08:07:56.7423565Z", // very important!!!
    "aboutPerson" : {
      "oid" : "D530141A-1E2A-4800-A118-3FC8D6EFE6D5"
    },
    "year" : "2026",
    "monthOfYear" : "8",
    "dayOfMonth" : "21",
    "hourOfDay" : "10",
    "minuteOfHour" : "20",
  }
}

```

```
    "durationMinutes" : 15,
    "subject" : {
      "ENGLISH" : "another appointment",
      "SPANISH" : "otra cita"
    },
    "urls" : [ {
      "id" : "main",
      "lang" : "BASQUE",
      "value" : "https://my-admin.eus/citizen-service-appointments/appointment345?lang=eu"
    },
    {
      "id" : "main",
      "lang" : "SPANISH",
      "value" : "https://my-admin.eus/citizen-service-appointments/appointment345?lang=es"
    } ]
  },
  "isNewOrUpdated" : true
}
]
```

4.2.2 [interop message] components

Using the previous sample, this section describes the details of the [interop message] fields

4.2.2.1 Interop Context

- The [interop context] for the **request** initiated by DENA-APP is:

```
"context" : {
  "message" : {
    "type" : "CLIENT_RETRIEVE_REQ",
    "correlationId" : "1AB7F413-399D-49F1-AC41-44D651E5799A",
    "interopRouteData" : [ {
      "denaComponentId" : "CLIENT_INSTALLMENT",
      "timestamp" : "2026-08-18T11:28:47.5237003Z"
    } ]
  },
  "originClientInstallment" : "8B5AE78A-7D42-4069-A626-959BB07276C5",
  "destinationAdmin" : {
    "id" : "admin-id",
    "oid" : "AFF5E893-2A7E-43C7-BB2B-1E7D1535306D"
  },
  "subjectPerson" : {
    "id" : "personid",
    "oid" : "D530141A-1E2A-4800-A118-3FC8D6EFE6D5"
  },
  "dataType" : {
    "id" : "data-type-id",
    "oid" : "2F92F808-6D5C-4A0F-AB68-3D4D03028CFE"
  }
}
```

What this [interop context] *says* about the [interop message] is:

- The [interop message] origin is a [client installment] (DENA-APP)
 - The [interop message] is for [data retrieving] and it's a [request]: **CLIENT_RETRIEVE_REQ**
 - A [correlation id] is maintained during ALL the [interop message] processing to help debug the system
 - The [interop message] left the [client installment] at 2026-08-18T11:28:47.5237003Z and has NOT passed through any other DENA component
- The [interop context] for the **response** returned to DENA-APP after the **request** was processed:

```
"context" : {
  "message" : {
    "type" : "CLIENT_RETRIEVE_RESP",
    "correlationId" : "1AB7F413-399D-49F1-AC41-44D651E5799A",
    "interopRouteData" : [
      {
        "denaComponentId" : "CLIENT_INSTALLMENT",
        "timestamp" : "2026-08-18T11:28:47.5237003Z"
      },
      {
        "denaComponentId" : "DENA_CORE",
        "timestamp" : "2026-08-18T11:28:47.6217025Z"
      },
      {
        "denaComponentId" : "DENA_ADMIN_CONNECTOR",
        "timestamp" : "2026-08-18T11:28:47.7315121Z"
      },
      {
        "denaComponentId" : "ADMIN",
        "timestamp" : "2026-08-18T11:28:47.7537244Z"
      },
      {
        "denaComponentId" : "DENA_ADMIN_CONNECTOR",
        "timestamp" : "2026-08-18T11:28:47.7722214Z"
      },
      {
        "denaComponentId" : "DENA_CORE",
        "timestamp" : "2026-08-18T11:28:47.8012511Z"
      }
    ]
  },
  "originAdmin" : {
    "id" : "admin-id",

```

```

    "oid" : "AFF5E893-2A7E-43C7-BB2B-1E7D1535306D"
  },
  "subjectPerson" : {
    "id" : "personid",
    "oid" : "D530141A-1E2A-4800-A118-3FC8D6EFE6D5"
  },
  "dataType" : {
    "id" : "data-type-id",
    "oid" : "2F92F808-6D5C-4A0F-AB68-3D4D03028CFE"
  }
}

```

What this [interop context] says about the [interop message] is:

- The [interop message] origin is an **[admin]** (*it's a response to a [data retrieve] request*)
- The [interop message] is for **[data retrieving]** and it's a **[response]: CLIENT_RETRIEVE_RESP**
- The **[correlation id]** is the same as the one in the original request from DENA-APP
- The [interop message] has gone through these DENA components:
 - o 2026-08-18T11:28:47.5237003Z > Left the [client installment]
 - o 2026-08-18T11:28:47.6217025Z > Processed at DENA-CORE
 - o 2026-08-18T11:28:47.7315121Z > The ADMIN-CONNECTOR handled the request
 - o 2026-08-18T11:28:47.7537244Z > The [admin] returned the requested [data]
 - o 2026-08-18T11:28:47.7722214Z > The ADMIN-CONNECTOR received the response
 - o 2026-08-18T11:28:47.8012511Z > DENA-CORE returned the [data] to the [client installment]

4.2.2.2 Payload

The [interop message] **payload** is what goes into JAVA-API method params or returned value

In the previous sample case, the JAVA-API business method was:

```

public <D extends DN00IsDENADDataExchangedObject>
    COREServiceMethodExecResult<DN00DataRetrieveResponse<D>> retrieveData(SecurityContext securityContext,
                                                                           DN00DataRetrieveRequest retrieveRequest);

```

And the **translations** a DENA-CORE REST-Service were:

- The **request's** [interop message] **payload** was **translated** to the JAVA-API method **params** (DN00DataRetrieveRequest)

```

"payload" : {
  "dataType" : {
    "id" : "citizen_service_appointments",
    "oid" : "BEDCB4AF-D384-4E05-B74E-A25D7322EF63"
  },
  "admin" : {
    "id" : "admin-id",
    "oid" : "D4348C40-84B8-4420-9747-193C75CB2875"
  },
  "person" : {
    "id" : "person-id",
    "oid" : "DAA35E71-5B28-44BF-9DAE-A412E1CEC538"
  },
  "clientInstallment" : "ED4576E0-DF47-4D2F-B039-A91228B3F09E",
  "lastSuccessfulClientRetrievalAt" : "2026-08-18T11:28:47.528701300Z"
}

```

- The JAVA-API method **return value** (DN00DataRetrieveResponse<DN00ScheduleItem>) was **translated** to the **response** [interop message] **payload**

```

"payload" : {
  "requestedNumberOfItems" : 0,
  "itemsPagingContext" : {
    "totalItemsCount" : 200,
    "startPosition" : 0,
  },
  "dataItems" : [
    {
      "data" : {
        "type" : "scheduleItem", // id used to handle polymorphic types (all [data] types implement DN00IsDENADDataExchangedObject
        "id" : "appointment345",
        "lastChangedAt" : "2026-08-19T08:07:56.7423565Z", // very important!!!
        "aboutPerson" : {
          "oid" : "D530141A-1E2A-4800-A118-3FC8D6EFE6D5"
        }
      }
    }
  ]
}

```

```
"year" : "2026",
"monthOfYear" : "8",
"dayOfMonth" : "18",
"hourOfDay" : "13",
"minuteOfHour" : "2",
"durationMinutes" : 30,
"subject" : {
  "ENGLISH" : "an appointment",
  "SPANISH" : "una cita"
},
"urls" : [ {
  "id" : "main",
  "lang" : "BASQUE",
  "value" : "https://my-admin.eus/citizen-service-appointments/appointment345?lang=eu"
},
{
  "id" : "main",
  "lang" : "SPANISH",
  "value" : https://my-admin.eus/citizen-service-appointments/appointment345?lang=es
} ]
},
"isNewOrUpdated" : false
}
...
]
}
```

5 DENA CONFIG

DENA is configurable in these areas:

- **Labeling system** that allows assets to be tagged with normalized [labels]
- **Organizational structure** of the [Admins] that are part of the system
- **Interop**: the [data types] supported by [dena] and the [data origins] that provide data for those [data types]

5.1 Labeling system

The [labelling] system contains [labels] that can be used to categorize any asset

Example: [data types] are categorized thematically and a [birth certificate] can be categorized with the [family] [label]

The system can potentially contain hundreds of [labels] of different [categories] so a mean to “select” the [labels] is needed. That mean to select [labels] is the **[taxonomy]** which is no more than a hierarchy disposition of [labels]:

```
+ social
  + family
  + young people
  + old people
  + inmigration
  + ...
+ culture
  + theatre
  + reading
  + ...
```

So, the **[taxonomy]** is **just a tool to select a [label]** which is what really matters: the [label] is what gets sticked to the categorized asset

A **[label]** la be view as a post-it that can be sticked to anything and the [taxonomy] as the tool for selecting the correct post-it

Being [labels] the most important object in [labelling] and [taxonomies] the tool used to “select” [labels], there exist other “auxiliary” object depicted below:

```
+ [domain group]
  + [domain]
    + [label]
    + [taxonomy group]
      + [taxonomy]
```

The **[domain group]** is a container of labeling **[domains]** which is also a container of:

- **[labels]**
- **[taxonomies]** of [labels] that can be grouped in **[taxonomy groups]**

5.1.1 Main Objects in the Labeling System

5.1.1.1 Label

A **[label]** represents a concept or a category and is composed by:

- a **unique identifier** (oid)
- **Standard [terms]** in different languages (just **one** standard [term] by language)
- **Synonym [terms]** in different languages (can exist **many** synonym [term] in a language)
- **External identifiers:** the concept / category that a [label represents] can exists in other systems where they usually have other [identifiers] different from the OID at the [labelling system]

That's all; no more, no less

5.1.1.2 Taxonomy

A **[taxonomy]** is a tool to select [labels] and have:

- unique identifier (oid)
- Name in different languages
- Hierarchical structure of [labels] like:

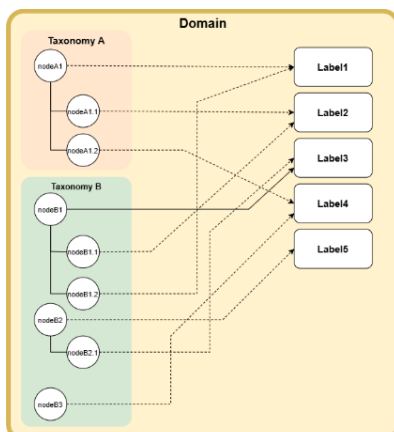
```
+ [label A]
  + [label A.1]
    + [label A.1.1]
    + [label A.1.2]
  + [label A.2]
    + [label A.2.1]
    + [label A.2.2]
+ [label B]
  + [label B.1]
    + [label B.1.1]
    + [label B.1.2]
+ [label A.1]
```

Note that [labels] can be repeated in a [taxonomy] as the sample of [label A.1]

Internally a **[taxonomy]** is represented as a **[node outline]** where every **[node]** is composed by:

- a **unique identifier**
- the **parent [node]** (if it's a root [node] the parent is null)
- **the [label] which this [node] links to** and the [domain] this [label] is contained in
Beware that [labels] are contained in a [domain] and so do [taxonomies] which in turn are contained in a [taxonomy group] which is also contained in a [domain]
... but a [taxonomy]'s [node] **can link to a [label] in another [domain]**
- the **[role]** of the [label] in the [taxonomy] (usually the [role] is just the depth of the [node] in the hierarchy but can by any string that like “theme” or “subtheme”)

Think the [node outline] as a hierarchical structure of [nodes] where every [node] is a **pointer** to a [label]



5.1.2 Java API

There is a java API to manage all these [model objects]: DN00ClientAPIForLabeling (api)

Here are some common operations:

1. Create a [domain group]

```
DN00LabelingDomainGroup domainGroup = DN00LabelingDomainGroupBuilder.create()
    .suppliedWithNewOid()
    .defaultGroup()
    .noDescription()
    .withNames(name)
    .build();

DN00LabelingDomainGroup savedDomainGroup = api.domainGroupAPI()
    .getForCRUD()
    .create(domainGroup);
```

2. Create a [domain] (remember that a [domain] is contained in a [domain group])

```
DN00LabelingDomain domain = DN00LabelingDomainBuilder.create()
    .suppliedWithNewOid()
    .noSecurityUID()
    .inGroup(domainGroupOid)
    .noDescription()
    .withNames(name)
    .build();

DN00LabelingDomain savedDomain = api.domainAPI()
    .getForCRUD()
    .create(domain);
```

3. Create a [label]

```
DN00Label label = DN00LabelBuilder.create()
    .withOid(DN00LabelOID.supply())
    .inDomain(domainOid)
    .enabled()
    .withStandardTerms(stdTerms)
    .noSynonimousTerms()
    .build();

DN00Label savedLabel = api.labelAPI()
    .getForCRUD()
    .create(label);
```

4. Create a [taxonomy group] (remember that a [taxonomy group] is contained in a [domain] which is in turn contained in a [domain group])

```
DN00TaxonomyGroup taxonomyGroup = DN00TaxonomyGroupBuilder.create()
    .suppliedWithNewOid()
    .asDefault()
    .inDomain(domainOid)
    .withNames(desc)
    .build();

DN00TaxonomyGroup savedTaxonomyGroup = api.taxonomyGroupAPI()
    .getForCRUD()
    .create(taxonomyGroup);
```

5. Create a [taxonomy] (remember that a [taxonomy] is contained in a [taxonomy group] which in turn is contained in a [domain] which also in turn is contained in a [domain group])

```
DN00Taxonomy taxonomy = DN00TaxonomyBuilder.create()
    .withOid(DN00TaxonomyOID.supply())
    .withBusinessIds(DN00TaxonomyID.forId("taxonomy"))
    .inDomain(domainOid)
    .inGroup(groupOid)
    .toBeUsedFor(DN00TaxonomyUsage.ALL)
    .withNames(desc)
    .build();

DN00Taxonomy savedTaxonomy = api.taxonomyAPI()
    .getForCRUD()
    .save(taxonomy);
```

6. Create [taxonomy] [nodes]

a) If it's a root node

```
DN00TaxonomyNode savedRootNode = api.taxonomyNodesAPI()
    .getForCRUD()
    .createRootNode(taxonomyOid,
        labelOid, domainContainingLabelOid,
        role);

// also can be created like:
DN00TaxonomyNode savedRootNode = api.taxonomyNodesAPI()
    .getForCRUD()
    .createNode(taxonomyOid,
        labelOid, domainContainingLabelOid,
        null, // root node
        role);
```

b) If it's not a root

```
DN00TaxonomyNode savedNode = api.taxonomyNodesAPI()
    .getForCRUD()
    .createNode(taxonomyOid,
        labelOid, domainContainingLabelOid,
        parentNodeOid,
        role);
```

[nodes] can also be created using builders:

```
DN00TaxonomyNode node = DN00TaxonomyNodeBuilder.create()
    .suppliedWithNewOid()
    .inTaxonomy(taxonomyOid).next()
    .childOf(parentNodeOid).next()
    .linkedToLabel(labelOid).from(domainContainingLabelOid)
    .actingAs(role)
    .build();

DN00TaxonomyNode savedNode = api.taxonomyNodesAPI()
    .getForCRUD()
    .createNode(node);
```

Once a [taxonomy] is created, usually a “view” of that [structure] containing data of

- the [structure] itself
- the [nodes] that composes the hierarchy
- the [labels] where the [nodes] point to

to load such [structure] view the java api is:

```
DN00TaxonomyView taxonomyView = _api.taxonomyNodesAPI()
    .getForNodeLabels()
    .usingCache()
    .loadTaxonomyView(taxonomy.getOid());
```

5.2 Org Config

The [org config] contains information about the [admins] that participate in DENA by offering [data]

The [org config] is organized in two levels:

```
+ [org groups]
  + [admins]
```

... so an [admin] always belongs to an [org group]

For example, the [org groups] can be:

```
+ [Administración General de la Comunidad Autónoma del País Vasco]
  + [Gobierno Vasco]
  + [Osakidetza]
  + [Lanbide]
  ...
+ [Administración Foral de Bizkaia]
  + [Diputación Foral de Bizkaia]
  ...
+ [Administración Local de Bizkaia]
  + [Ayuntamiento de Bilbao]
  + [Ayuntamiento de Barakaldo]
  + [Ayuntamiento de Getxo]
  ...
+ [Administración Foral de Gipuzkoa]
  + [Diputación Foral de Gipuzkoa]
  ...
+ [Administración Local de Gipuzkoa]
  + [Ayuntamiento de Donostia-San Sebastian]
  + [Ayuntamiento de Irún]
  ...
+ [Administración Foral de Araba]
  + [Diputación Foral de Arba]
  ...
+ [Administración Local de Araba]
  + [Ayuntamiento de Vitoria-Gasteiz]
  ...
```

An [org group] is composed of:

- A unique identifier (oid / uid)
- The name in multiple languages

An [admin] is composed of:

- A unique identifier (oid / uid)
- The [org group] to which the [admin] belongs to (*parent [org group]*)
- The name in multiple languages
- Sync and Retrieve config:
 - **Priority for [data retrieval] at DENA-APP:** If the SRMD SYNC phase reveals a large number of [data origins] (admins) to retrieve from, DENA-APP can **prioritize background retrieval** tasks (even though these retrievals typically execute in parallel)
 - **Cold Start [data types]:** when a [person] starts using DENA (*the first time he/she enters DENA*), no [admin] have sent any SRMD so DENA do NOT know anything about the [person] and the [data] she/he has in any [admin], and the [client app] cannot show anything.

To overcome this problem, when a [person] starts using DENA, the system **forces the [app] to retrieve [data] from certain [admins]/[data types]** by inserting SRMD as if it was the [admin] who has sent it in the [meta-data sync]

This way, the [client app] sees that it must retrieve [data] from certain [data origins] ([*admin*] – [*data type*]) and at least [data] from *relevant* [admins] is shown in the [app]

5.3 Interop Config

The [interop config] has two main objects:

- Data Type: the different kinds of [data] that the [admins] offers to DENA
- Data Origin: how the [data] is offered by the [admins] through [data providers]

5.3.1 Data Type

A [data type] is a simple config object that just contains:

- The **name** in different languages
- The [labels] in which the [data type] is categorized
These [labels] are used by the [app] (mobile / web) to know what [data types] belongs to a certain [theme] (category) so it can “paint” the UI

5.3.2 Data Origin

The [data origin] config is used internally by DENA-CORE to know how to *talk* with the [data provider] service exposed at the [admins]. **It’s DENA’s router**

To understand the [data origin] config, a basic understanding of the DENA connector architecture is needed:

A DENA [connector] is a component of DENA-CORE that intermediates with the [data provider] services exposed at the [admins].

[DENA-CORE] <----> [DENA-CONNECTOR] <----> [ADMIN]

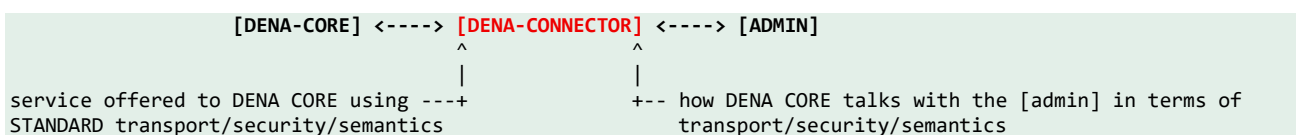
To achieve this intermediation work, the [connector] have to deal with three areas:

- [transport] (http endpoints) > how to connect to the [data provider] at the [admin]
- [security] (OAUTH, X509, usr/pwd, etc) > how to authenticate at the [data provider] at the [admin]
- [semantics] (the data format) > how to *understand* the [data] returned by the [admin].

DENA defines a **standard semantic** for every [data type] and if the [admin] adheres to that standard in the [data provider], the [connector] is straightforward; it has nothing to do.

... but if the [admin] for example returns the [data] in a different format than the standard, the [connector] must **translate** from what the [admin]’s [data provider] returns to **standards**

A DENA [connector] at a very high level is like:



- DENA-CORE *talks* with the **standard semantics** with the **internal side** of the [connector]
In this case, just the [transport] config is needed since the [security] and [semantics] are standardized for the **internal side** of all [connectors]
- The [connector] **external side** *may talk* with **different semantics** with the [data provider] service offered by the [admin]
In this case, the [connector] needs to know how to *talk* to an [admin] in terms of:
 - [transport] ([http endpoints](#))
 - [security] (OAUTH, X509, usr/pwd, etc)
 - [semantics] (the data format)

The concept that encapsulates ALL this information is the [connector config]

The [data origin] config structure is like:

```
+ [data origin]
+ (N) [data type] config..... the [data types] (EACH [data type] have it's OWN [connector config]))
+ [connector config]..... how to CONNECT to RETRIEVE the [data]
+ (1) EXTERNAL side config ..... how the [connector] calls the [admin] remote party
+ (N) [admin]..... the [admins] that is [connector config] for a [data type] applies to
                    (the SAME [connector config] might be used for several [admins] because a
                     SINGLE [data provider] service provides [data] for them all)
+ (1) INTERNAL side config > how DENA-CORE calls the [connector] .....how DENA-CORE connects to the INTERNAL SIDE of the
                    [connector] (a [data origin] just have a SINGLE INTERNAL
                    [connector config] for ALL [data types])
```

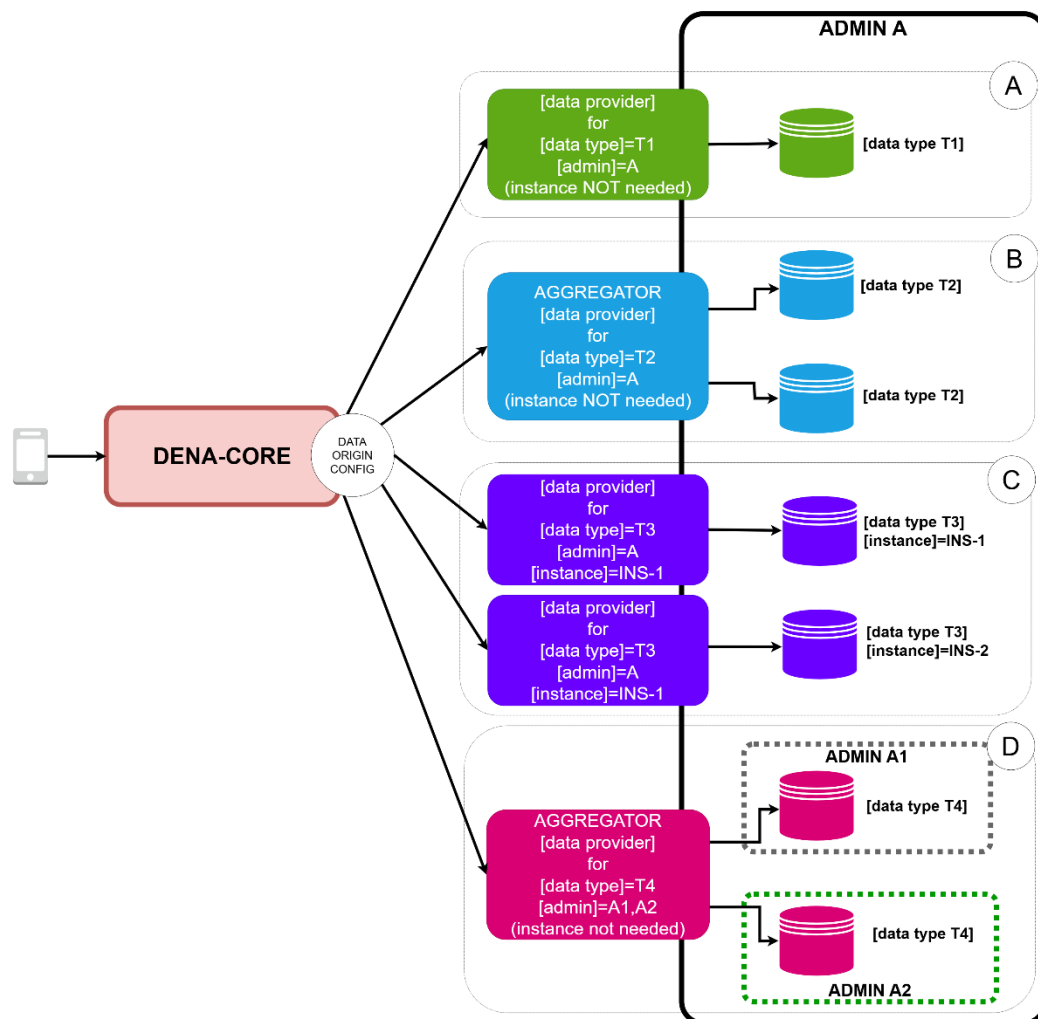
So, a [data origin] is composed of:

- 1 config used by DENA CORE to know how to *talk* with the **internal side** of the [connector]
- N configs, one for each **[data type] + [data origin instance]** that sets how the [connector] *talks* to each remote [data provider]

The [data origin] config in essence is a **routing table** for the [data provider] at the [admins] using:

- The [admin]
- The [data type]
- The [data origin instance]

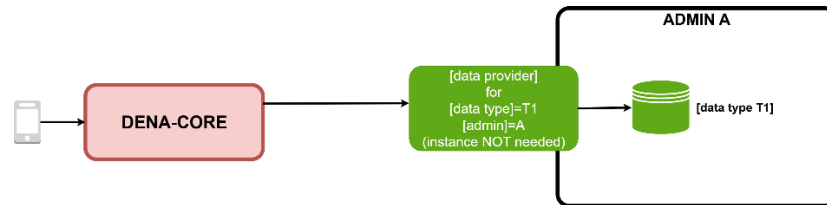
[Admins] can offer [data] through [data providers] using different **architectures** as shown in the following image:



The options are:

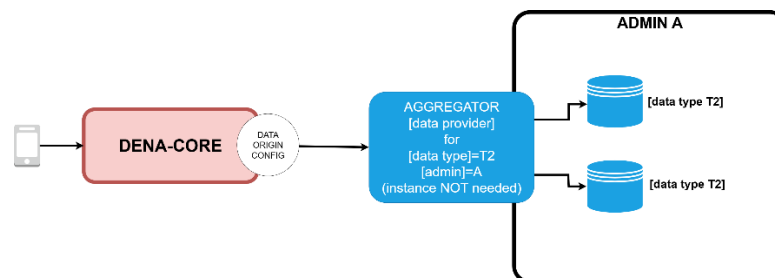
- a) A [data provider] for a **single [data type] in an [admin]**

At the [admin] side, a **dedicated** [data provider] offers the [retrieval service] for the [data type] in the [admin]

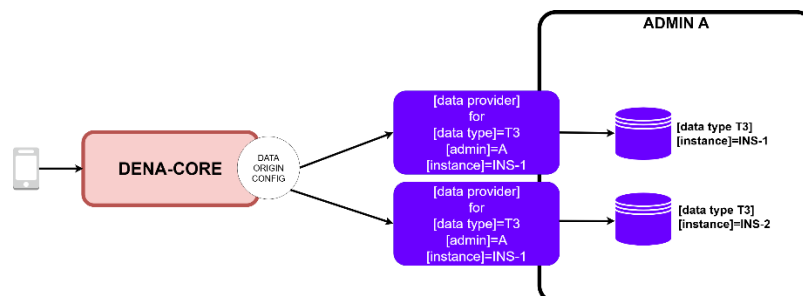


- b) A [data provider] that **aggregates [data]** of a certain [data type] from **multiple origins at different [instances] of a system** (for example, multiple [procedure record management systems] -gestores de expedients-)

This is the case of [admins] that have aggregated multiple [instances] in a common [data lake] which acts as the [data source] of it's [citizen view]

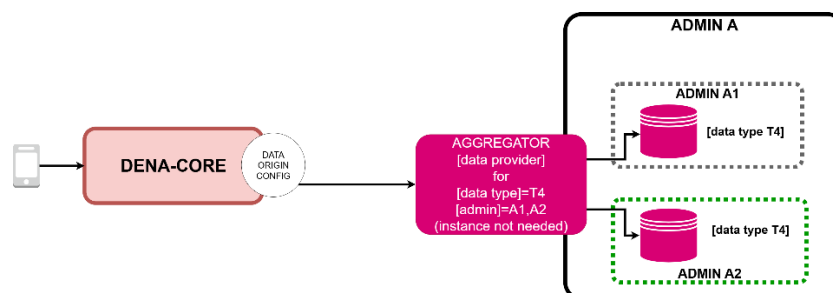


- c) **Multiple [data providers] for multiple origins at different [instances] of a system** (for example, multiple [procedure record management systems] -gestores de expedients-)



- d) A [data provider] that **aggregates [data]** of a certain [data type] from **multiple origins in different [admins]**

This is the case of supra-territorial orgs like [diputaciones forales] that provides [data] in the SAME [data provider] for multiple [ayuntamientos]



5.4 AppVersion Config

DENA-CORE stores historical data about **[app] (mobile / web) versions** in terms of:

- The **[version number]** with the pattern {major}.{minor}.{patch}[-alias] (alias is optional)
- Whether the [version] is a **developer preview**
- The [version] **life range** (start..end) *-if the end bound is null, it's the current version, so just one [version] without upper bound should be registered in the system-*
- The [version] **status**: PERMANENT / TEMPORARY DISABLED or ENABLED
- The target **[client platform]**: MOBILE_ANDROID / MOBILE_IOS and WEB_BROWSER
- **Messages** to be shown to the user after a [version] update

The [app version] config is used mainly in the **init phase** of the [client app] when the [client app] sends DENA-CORE data about the installed [app version].

With this [app version] and the latest available [app version], DENA-CORE can instruct the [client app] to:

- Completely **disable** de [app] because the installed [app version] is no longer usable or temporary unavailable
- Force the [client] to **update** the [app version]

... more about this in the **[client app] init documentation**

6 BASIC DENA DATA-TYPES

DENA uses a set of **[model objects]** that constitutes one of the cornerstones of the system.

This section describes the basic [model objects] that are worth being aware of:

6.1 Foundational data-types

6.1.1 Boolean

true / false (*always minus*).

6.1.2 Numbers

Los números pueden ser:

Type	Description	Sample
Integer	Integer number (<i>natural number without decimals</i>) with sign. A 64 bits representation is used (Java Long). Range: [-9.223.372.036.854.775.808 .. 9.223.372.036.854.775.807]	25
FloatingPoint	Double precision floating point number A 64 bits representation is used (Java Double). Range: [4.9E-324 .. 1.8E+308]	12.5 Point is used as decimal separator

6.1.3 Enums

[enum] represent values in a pre-defined finite spectrum (value set). They are serialized as a text string always in UPPERCASE

ACTIVE / INACTIVE

6.1.4 Language / Country

[Language] is an specialized [Enum] with languages supported by DENA:

Enum Val	Int code	iso639_1	iso639_2
SPANISH	10	es	spa
BASQUE	11	eu	eus
ENGLISH	20	en	eng
FRENCH	30	fr	fra
DEUTCH	40	de	deu
ITALIAN	58	it	ita
PORTUGUESE	59	pt	por
...			

Enum	Val	iso3166_1	iso3166_2
SPAIN	724	ES	ESP
UNITED_KINGDOM	826	GB	GBR
FRANCE	250	FR	FRA
GERMANY	276	DE	DEU
ITALY	380	IT	ITA
PORTUGAL	620	PT	PRT
...			

6.1.5 String

A text

Sample text string

6.1.6 LanguageTexts

A kind of Map<Language,String> whose [key] is the [language] and the [value] a text in that [language]

- **BASQUE**: a text in [basque].
- **ENGLISH**: a text in [english].
- **SPANISH**: a text in [spanish]

```
{
  "SPANISH": "Expediente",
  "BASQUE": "Espediente",
  "ENGLISH": "Administrative Record"
}
```

6.1.7 Date Time

Date Time can be:

Type	Description	Representation
LocalDate	Date without time and [time zone] Format: yyyy-MM-dd	2024-12-15
UTCDateTime	Date and time with millisecond accuracy Always UTC. Format: yyyy-MM-ddTHH:mm:ss.SSSZ (Z=UTC time zone) [ISO 8601 Instant] (UTC) format with millisecond accuracy	2023-12-16T13:00:00.000Z
OffsetDateTime	Date and time with millisecond accuracy in a certain [time zone] Format: yyyy-MM-ddTHH:mm:ss.SSSXXX (XXX=time zone) [ISO 8601] format with millisecond accuracy	2026-02-05T14:30:15.123+01:00
TimeStamp	An [instant], a point in time always in the UTC [time zone] [EPOCH TimeStamp] is used which is the number of seconds from a remote point in time: 1970-01-01T00:00:00Z	1670374400 (epoch TimeStamp for 2023-12-16T13:00:00.000Z 1670374400 seconds from 1970-01-01T00:00:00Z)

Another util types related with date-time:

Type	Description	Representation
TimeLapse	Period of time serialized as: {number}{d h m s ms us ns}	5d = 5 days 3s = 3 seconds
TimeFrequency	An enum with the following values: – DAYLIY – WEEKLY – MONTHLY – QUARTERLY – YERARLY	2023-12-16T13:00:00.000Z

6.1.8 Ranges

Ranges represent a discrete interval (*finite number of values*) of numbers or dates with two bounds (*boundaries*) that can be or can not be included in the range

Range bounds are represented/serialized as:

- A [or] character if the bound value is included in the range.
- A (or) character if the bound value is NOT included in the range.

Examples:

[1..20]	A number between 1 and 20 both 1 and 20 included	$X \geq 1 \ \&\& \ X \leq 20$
[1..20)	A number between 1 and 20, 1 included and 20 excluded	$X \geq 1 \ \&\& \ X < 20$
(1..20]	A number between 1 and 20, 1 excluded and 20 included	$X > 1 \ \&\& \ X \leq 20$
(1..20)	A number between 1 and 20 both 1 and 20 excluded	$X > 1 \ \&\& \ X < 20$
[1..] o [1..)	Greater or equal to 1	$X \geq 1$
(1..] o (1..)	Greater to 1	$X > 1$
[..20] o (..20]	Less or equal to 20	$X \leq 20$
[..20) o (..20)	Less than 20	$X < 20$

NOTE: in DENA interoperability services, **by default inclusive bounds will be used**

More samples with most common Ranges

	Representation
Local Date Range (date format yyyy-MM-dd)	[2023-01-01..2023-12-31]
DateTime Range (ISO 8601 instant)	[2023-01-01T00:00:00.000Z..2023-01-01T13:00:00.000Z]
Integer or floating point ranges	[12..15] [1.5..5.5]

6.1.9 UID / OID

An UID (Unique Identifier) and it's specialization OID (Object identifier) are a **unique universal identifier without business meaning** randomly generated in a way that the odds of two equal UIDs are minimal

UIDs / OIDs do not repeat

UIDs / OIDs are represented as a 36 hex string organized in 5 groups of 8 characters separated by the character '-'

Sample `db761b72-1634-4fb0-b7f1-3c1ebdbb1eb`

Sometimes **SHORT UIDs** are used which are simply UIDs with a shorter length which implies that the odds collision between two [short UIDs] cannot be ignored

[short UIDs] will be used in special situations where a normal (long) UID is not necessary because the set of objects to be assigned with an UID is very small or to name short-live objects (*i.e. an expiring message*)

Sample `db761b72`

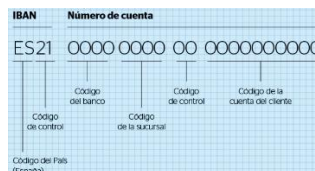
6.1.10 ID

IDs are **identifiers with business meaning** like:

- A [person] identifier (*ie. An spanish NIF*).
- A [administrative record] number (*ie: EXP-2023-0004232*).
- The [IBAN] number that identifies a [bank account]
- Etc.

IDs usually have an structure (*a format / pattern*) as the case of:

- [IBAN] (International Bank Account Number) used to identify [bank accounts] in the SEPA zone = Single Euro Payments Area.



Sample `ES2199999111100999999999`

- Spanish NIF (*Numero de identificación Fiscal*) which is an umbrella for several types of tax identification numbers in Spain:
 - DNI (Documento nacional de identidad): for spanish citizens > 8 digits + 1 letter
 - NIE (Numero de Identidad Extranjero): for foreign residents in Spain > 1 letter (X / Y / Z) + 7 digits + 1 letter
 - CIF (Código de Identificación Fiscal): for legal entities (companies, associations, etc) > 1 letter (A, B, C, D, E, F, G, H, J, U, V) + 7 digits + 1 control character (letter or digit)
 - NAF (Número de Afiliación a la Seguridad Social): for social security purposes

6.1.11 Token

Tokens are similar to [ID] but generally related to security

Sample `myToken`

6.1.12 Urls

6.1.12.1 Url

Model san URL with no more info besides::

- Protocol (http / https)
- Port
- Host
- UriPath
- QueryString
- Anchor

Sample

```
https://dena.eus/payments#concepts?param1=a&param2=b
```

- | | |
|-------------------------|----------------------------------|
| - Protocol: https | - uriPath: /mipago |
| - Port: 80 | - queryString: param1=a¶m2=b |
| - Host: www.euskadi.eus | - anchor=concepts |

6.1.12.2 UrlCollection

A collection (a bunch of) [Url]s where some attributes can be set for every collection item (*for every collection's [Url]*):

- An ID
- The language
- Tags

Sample:

```
[
  {
    "id": "mipago",
    "url": "https://www.euskadi.eus/nireordainketa",
    "lang": "BASQUE"
  },
  {
    "id": "mipago",
    "url": "https://www.euskadi.eus/mipago",
    "lang": "SPANISH"
  }
]
```

6.1.12.3 WebLink

Models a [web link] -an url in a web page-:

- Url Text (*what's seen in the web page*)
- Language
- Presentation attributes

Sample:

```
{
  "url": "https://www.euskadi.eus/nireordainketa",
  "texts" {
    "lang": "BASQUE",
    "title": "Euskal administrazioen ordainketa-pasabidea",
    "description": "Admin baten edozein likidazio ordaintzeko aukera ematen du",
  }
}
```

6.1.12.4 WebLinkCollection

A collection of [WebLinks]

Sample:

```
[
  {
    "url": "https://www.euskadi.eus/mipago",
    "texts" {
      "lang": "SPANISH",
      "title": "Pasarela de Pagos de las Administraciones Vascas",
      "description": "Permite pagar cualquier liquidacion de una admin",
    }
  },
  {
    "url": "https://www.euskadi.eus/nireordainketa",
    "texts" {
      "lang": "BASQUE",
      "title": "Euskal administrazioen ordainketa-pasabidea",
      "description": "Admin baten edozein likidazio ordaintzeko aukera ematen du",
    }
  }
]
```

6.1.13 Money

Represents an ammount of money in currency expressed from a base

Field	Description
base* <i>Enum:</i> - CENTS - UNITS	Currency smallest unit
currency* <i>String</i>	Currency code
amount* <i>Integer</i>	Ammount.

Sample:

```
{
  "base": "CENTS",
  "currency": "EUR",
  "amount": 249333
}
```

6.1.14 Hash (digest)

Digest of a text computed applying [SHA-256] algorithm with the following features:

- **Hash fixed format and length** no matter the input [text] in terms of length or content
- **Consistency:** the same input [text] always yields to the same [hash]
- **Unidirectional:** is nearly impossible to guess / reconstruct the original input [text] from the [hash]
- **Collision Resistant:** is nearly impossible that two different input [text] yield to the same [hash]
- **Extreme sensibility:** a minor change in the input [text] yields to a completely different [hash]

Sample:

Original Text

Hash

This is a random text to be hashed 2251f5ac60e4fc24a62914bf92a9adf3af1abbf72d649eceec9ed7c3a2e29b8f

6.1.15 UserAgent

[User-Agent] is an string that is included in DENA interop messages as an HTTP header that provides info about the **cliente device** (web browser, mobile app, etc.)

There's NO standard for the [User-Agent] header, but at least a pattern is generally used (*although the pattern can be very different between client devices*)

The [User-Agent] most common pattern is:

<product>/<version> (<system-information>) <platform> (<platform-details>) <extensions>

For example the [User-Agent] sent by [Chrome] [web browser] in [Android] is:

Mozilla/5.0 (Linux; Android 16; Pixel 9) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/143.0.12.45 Mobile Safari/537.36

<product>/<version>	For compatibility reasons [web browsers] always send [Mozilla/5.0]
<system-information>	Linux; Android 16; Pixel 9
<platform>	AppleWebKit/537.36
<platform-details>	KHTML, like Gecko
<extensions>	Chrome/143.0.12.45
	Mobile
	Safari/537.36

In [DENA] the [User-Agent] [header] depends on which is sending the message:

- If the [mensaje] is **sent by the [client device]** (*mobile app / web app*):

MobileApp

DENA-APP is a [Flutter] app so the [User-Agent] format is:

**DenaApp/<DenaApp ver> (<system> <System ver>; <device> <device ver>) Dart/<dart versión>
Flutter/<flutter ver>**

Sample: DenaApp/1.0 (Android 13; Pixel 6 Pro) Dart/3.0 Flutter/3.16

App Web

A [web browser] [User-Agent] will be determined by the [web browser]

For example, [Chrome] in a [Mac (Intel)] platform:

Mozilla/5.0	(Macintosh;	Intel	Mac	OS	X	10_15_7)
AppleWebKit/537.36		(KHTML,		like		Gecko)
Chrome/143.0.0.0	Safari/537.36					

- If the message is sent by DENA-CORE

DENA-CORE/<dena-core ver> <module>/<module ver> (support@dena.eus)

For example the [person-data] module can "notify" an [Admin] that a new [person] has enrolled DENA sending an HTTP message to an [admin] *listener*

The [user-agent] in this HTTP call would be like:

DENA-CORE/2.1.0 person-data/1.0.1 (support@dena.eus)

- If the message is sent to DENA-CORE by a third party (*an [admin]*) the [user-agent] will be like:

AdminX/<AdminX ver> <module>/<module ver> (support@adminX.eus)

For example, if an [admin] "notifies" DENA-CORE of [changes] in a [person]'s [data] (SRMD SYNC send) a message is sent to a DENA-CORE HTTP *listener* and the [user-agent] is:

AdminX/1.1.0 expedientes/1.0.0 (support@adminX.eus)

The [user-agent] [header] is NOT mandatory but it's very useful for debugging purposes